

DECENTRALIZED BLOOD DONOR APPLICATION

A PROJECT REPORT

Submitted by

SARAVANAKUMAR S (421622244301)

LOKESH V (421622244024)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND BUSINESS SYSTEMS

MAILAM ENGINEERING COLLEGE

MAILAM



ANNA UNIVERSITY: CHENNAI 600 025

MAY 2026



DECENTRALIZED BLOOD DONOR APPLICATION



A PROJECT REPORT

Submitted by

SARAVANAKUMAR S (421622244301)

LOKESH V (421622244024)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND BUSINESS SYSTEMS

MAILAM ENGINEERING COLLEGE, MAILAM

ANNA UNIVERSITY: CHENNAI 600025

MAY 2026

ANNA UNIVERSITY: CHENNAI 600025

BONAFIDE CERTIFICATE

Certified that this project report “ **DECENTRALIZED BLOOD DONOR APPLICATION A Secure and Transparent Platform Using Blockchain** ” is the Bonafide work of “**SARAVANAKUMAR S (421622244301) and LOKESH V (421622244024)**” who carried out the project work under my supervision.

SIGNATURE

Dr. S. PRASANNA, M.E., Ph.D.,

HEAD OF DEPARTMENT

Professor & Head,
Department of Computer Science
And Business Systems
Mailam Engineering College
Mailam – 604304

SIGNATURE

Dr. S. PRASANNA, M.E., Ph.D.,

SUPERVISOR

Assistant Professor,
Department of Computer Science
And Business Systems
Mailam Engineering College
Mailam – 604304

Submitted for the Project Viva – Voce Examination held on.....

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We praise our college founder **Shri. N. KESAVAN** and we thank our Chairman **Shri. M. DHANASEKARAN**, secretary **Dr. K. NARAYANASWAMY** and Treasurer **Shri. D. RAJARAJAN** and Joint Secretary **Shri. S. VELAYUDHAM** for providing us with sufficient facilities and good environment.

We express our thanks to our Director **Dr. S. SENTHIL, M.Tech., Ph.D.**, and Principal **Dr. R. RAJAPPAN, M.E., Ph.D.**, has been motivation force to all our needs.

It is our privilege to express profound gratitude to our Project Coordinator and Professor & Head of the Department of Computer Science and Business Systems **Dr. S. PRASANNA, M.E., Ph.D.**, for his constant inspiration to our project work.

We express our sincere gratitude to our Batch Coordinator **Mrs. G. RAMYA, M.Tech., Assistant Professor**, for her constant guidance with our project work.

We express our sincere thanks to our internal guide **Mrs. G. RAMYA, M.Tech., Assistant Professor** Department of Computer Science and Business Systems for her timely advice, encouragement and unbelievable sense of commitment in guiding us to bring out this project with great success.

We convey our special thanks to Staff members and Lab assistants in our college for their valuable suggestions and hearty encouragement throughout the completion of our project work.

We thank our most beloved **Parents and Friends** whose continuous support and encouragement to pursue the degree and confidently complete the project.

ABSTRACT

This project presents a decentralized blood donation management system that integrates blockchain technology and token-based incentives to enhance transparency and donor engagement in the blood donation ecosystem.

The system uses the Aptos blockchain with Move smart contracts to store donor records on-chain and a geo-based matching algorithm to connect compatible donors and recipients based on blood group and location.

By combining these two methods, the system reduces blood wastage and increases donation reliability compared to traditional centralized blood bank management systems.

The system operates continuously and provides immediate rewards using Blood Donor Coins (BDC) and NFT Donation Certificates whenever a successful blood donation is confirmed.

It is designed to be cost-effective, transparent, and suitable for real-world applications across hospitals, blood banks, and veterinary care institutions through the Dogin animal donation module.

The proposed system can significantly reduce blood shortages caused by poor donor engagement and improve healthcare equity through a community-driven, blockchain-powered donation ecosystem.

திட்டப்பணிச்சுருக்கம்

இந்தத் திட்டம் ஒரு பரவலாக்கப்பட்ட இரத்த தான மேலாண்மை அமைப்பை முன்வைக்கிறது. இது இரத்த தான சுற்றுச்சூழலில் வெளிப்படைத்தன்மையையும் நன்கொடையாளர் ஈடுபாட்டையும் மேம்படுத்த சங்கிலி-தொகுப்பு தொழில்நுட்பம் மற்றும் அடையாள-நாணய அடிப்படையிலான ஊக்கங்களை ஒருங்கிணைக்கிறது.

இந்த அமைப்பு ஆப்டோஸ் சங்கிலி-தொகுப்பை நகர்வு-மொழி அறிவார்ந்த ஒப்பந்தங்களுடன் பயன்படுத்தி நன்கொடையாளர் பதிவுகளை தொகுப்பில் சேமிக்கிறது. மேலும் இரத்த வகை மற்றும் இடம் ஆகியவற்றின் அடிப்படையில் பொருத்தமான நன்கொடையாளர்களையும் பெறுநர்களையும் இணைக்க புவி-அடிப்படையிலான பொருத்தல் கணக்கீட்டு முறையைப் பயன்படுத்துகிறது.

இந்த இரண்டு முறைகளையும் இணைப்பதன் மூலம், பாரம்பரிய மையப்படுத்தப்பட்ட இரத்த வங்கி மேலாண்மை அமைப்புகளுடன் ஒப்பிடும்போது இரத்த வீணாவதை கணிசமாகக் குறைத்து தான நம்பகத்தன்மையை அதிகரிக்கிறது.

ஒரு வெற்றிகரமான இரத்த தானம் உறுதிப்படுத்தப்படும் போதெல்லாம், இந்த அமைப்பு தொடர்ச்சியாக இயங்கி இரத்த-தானியர் நாணயங்கள் மற்றும் மாற்ற இயலாத டிஜிட்டல் சான்றிதழ்களைப் பயன்படுத்தி உடனடி வெகுமதிகளை வழங்குகிறது.

இது செலவு குறைந்ததாகவும், வெளிப்படையானதாகவும் மற்றும் விலங்கு தான தொகுதி மூலம் மருத்துவமனைகள், இரத்த வங்கிகள் மற்றும் கால்நடை மருத்துவ நிறுவனங்களில் நிஜ உலக பயன்பாடுகளுக்கு ஏற்றதாகவும் வடிவமைக்கப்பட்டுள்ளது.

முன்மொழியப்பட்ட இந்த அமைப்பு, மோசமான நன்கொடையாளர் ஈடுபாட்டால் ஏற்படும் இரத்தப் பற்றாக்குறையை கணிசமாகக் குறைக்கலாம். மேலும் சமூக சார்ந்த, சங்கிலி-தொகுப்பு இயங்கும் தான சுற்றுச்சூழல் மூலம் சுகாதார சமத்துவத்தை மேம்படுத்தலாம்.

TABLE OF CONTENTS

CHAPTER NO NO	TITLE	PAGE
	ABSTRACT	iv
	ABSTRACT IN TAMIL	v
	LIST OF ABBREVIATIONS	ix
	LIST OF FIGURES	x
1.	INTRODUCTION	11
2.	OBJECTIVES	13
4.	LITERATURE SURVEY	14
4.	METHODOLOGY	16
	4.1 Overall Working Principle	
	4.2 Geospatial Donor Matching	
	4.3 Blockchain-Based Verification	
	4.4 Data Processing and Decision Making	
	4.5 Alert and Notification Mechanism	
	4.6 System Architecture	
	4.7 Real-Time Operation	
	4.8 Advantages of the Methodology	
	4.9 AI Analyser Module	
5.	SYSTEM ARCHITECTURE	23
	5.1 Frontend	
	5.2 Backend	

	5.3 Database	
	5.4 AI Analyst	
	5.5 Maps	
	5.6 Security	
6.	TOOLS & TECHNOLOGIES USED	26
	6.1 React.js	
	6.2 Node.js and Express.js	
	6.3 MongoDB	
	6.4 Ethereum Blockchain (Sepolia Testnet)	
	6.5 Ethers.js	
	6.6 Cloudinary	
	6.7 Nodemailer and Gmail SMTP	
7.	SYSTEM IMPLEMENTATION	29
	7.1 Frontend Implementation	
	7.2 Backend Implementation	
	7.3 Blood Request and Donor Matching	
	7.4 Donation Lifecycle Management	
	7.5 Inventory Management	
	7.6 Blockchain Data Processing	
	7.7 Decision-Making Algorithm	
	7.8 Notification System Implementation	
8.	RESULTS AND DISCUSSION	36
	8.1 Experimental Setup	
	8.2 Observations	
	8.3 Multi-Modal Detection Performance	
	8.4 Notification System Performance	
	8.5 System Accuracy and Reliability	
	8.6 Discussion	
	8.7 AI Analyser Performance	

9.	ADVANTAGES AND LIMITATIONS	41
10.	FUTURE SCOPE	45
11.	SOURCE CODE	46
12.	SCREENSHOT	93
13.	REFERENCE	96
14.	CONFERENCE	97

LIST OF ABBREVIATIONS

DApp	-	Decentralized Application
DBC	-	Blood Donor Coins
NFT	-	Non-Fungible Token
JWT	-	JSON Web Token
API	-	Application Programming Interface
REST	-	Representational State Transfer
RBAC	-	Role-Based Access Control
SHA	-	Secure Hash Algorithm
AI	-	Artificial Intelligence
EHR	-	Electronic Health Record
GIS	-	Geographic Information System (via MongoDB 2dsphere)
SMTP	-	Simple Mail Transfer Protocol

CDN	-	Content Delivery Network
UI	-	User Interface
ODM	-	SimObject Document Mapper

LIST OF FIGURES

FIGURE NO		FIGURE NAME
Fig 1	-	Blood Inventory
Fig 2	-	Donor Dashboard
Fig 3	-	Map View
Fig 4	-	Blockchain Recording
Fig 5	-	Admin Panel
Fig 6	-	AI Analyst Dashboard
-	-	System Architecture of BloodLink Blood Donation Platform

CHAPTER 1

INTRODUCTION

Blood shortage is one of the major causes of preventable deaths worldwide, posing a serious threat to both patients in critical care and the healthcare systems that serve them. Insufficient blood supply reduces a hospital's ability to perform surgeries, manage trauma cases, and treat chronic conditions such as anaemia and cancer. According to various healthcare studies, a significant percentage of emergency deaths occur due to unavailability of compatible blood, especially during mass casualty events, natural disasters, or in remote and underserved regions. Unlike equipment shortages, blood scarcity is a human participation problem that is difficult to resolve without the support of intelligent and incentivized systems.

Traditional blood donation systems such as blood camp drives, manual donor registries, and centralized blood bank databases are designed to collect and distribute blood but do not address the root cause of donor disengagement. Therefore, there is a growing need for advanced platforms that can connect donors and recipients in real time and provide meaningful recognition before critical shortages occur. With the advancement of blockchain technology, decentralized applications, and token-based incentive systems, it has become possible to develop solutions that continuously engage donors and transparently track every unit of donated blood.

Existing blood donation management systems typically rely on a single approach, either a centralized hospital database or a manual camp registration system. Centralized systems store donor records in institutional databases managed by

hospitals or government agencies. While these systems are structured and familiar, they are vulnerable to data breaches, lack cross-institutional visibility, and offer donors no transparency into how their blood is used. Decentralized approaches, on the other hand, have been explored in research but lack practical implementation with real incentive mechanisms and user-friendly interfaces.

To overcome these limitations, this project proposes BloodLink — a decentralized blood donation DApp built on the Aptos blockchain that combines on-chain donor management with a geo-based matching system. The system uses Move smart contracts to store donor profiles and hospital blood requests immutably on the blockchain. At the same time, a geo-based algorithm is used to identify nearby compatible donors, and token-based incentives are applied to reward every confirmed donation with Blood Donor Coins (BDC) and NFT Donation Certificates. By integrating these approaches, the system improves donor engagement and reduces blood wastage significantly.

The proposed system is designed to operate continuously, providing immediate rewards to donors through BDC tokens and verifiable NFT certificates when a successful blood donation is confirmed. It is transparent, cost-effective, and suitable for implementation across hospitals, blood banks, and veterinary care institutions through the Dogin animal donation module. This project contributes to the development of intelligent healthcare ecosystems by providing a reliable and community-driven solution for reducing blood shortages caused by poor donor participation and lack of transparency.

CHAPTER 2

OBJECTIVES

1. To develop a real-time blood donation management system that continuously monitors blood availability and connects donors with hospitals in need.
2. To integrate both blockchain-based verification and geospatial matching for more accurate and transparent donor-recipient coordination.
3. To minimize blood wastage and fulfillment delays by adopting a multi-modal approach combining inventory management, geo-matching, and automated expiry handling instead of relying on a single method.
4. To design a non-invasive and user-friendly platform that ensures donor comfort and does not interfere with normal daily activities while encouraging regular participation.
5. To implement a reliable incentive mechanism using Blood Donor Coins (BDC) and NFT donation certificates to reward donors immediately when a donation is successfully completed.
6. To ensure the system is cost-effective and easily deployable for both small clinics and large hospitals as well as for veterinary blood donation through the Dogin module.
7. To build a scalable and extensible system architecture that supports future enhancements such as cross-chain blockchain integration, government blood bank API connectivity, and mobile application development.

CHAPTER 3

LITERATURE SURVEY

Blood donation management has been an active area of research due to its critical role in saving lives. Various approaches have been proposed over the years, broadly categorized into **centralized blood bank systems**, **blockchain-based healthcare systems**, and **token incentive-based platforms**. Each approach has its own advantages and limitations.

Centralized blood bank systems such as eRaktaKosh (India) primarily rely on web-based databases to manage blood stock and donor records. These systems store information such as **blood group, donor details, location, and availability status**. One commonly used feature is manual inventory tracking, which helps hospitals identify available blood units. While these systems are easy to administer, they are highly vulnerable to data tampering, lack real-time donor-recipient matching, and offer no transparency to donors regarding blood usage.

Blockchain-based healthcare systems, on the other hand, focus on maintaining **immutable, decentralized records** of medical transactions such as **Electronic Health Records (EHR), drug supply chain tracking, and clinical trial data**. These methods provide direct auditability and transparency into the healthcare process and are generally more reliable in preventing data fraud. However, they

often require technical expertise to implement, involve transaction gas fees, and are not always practical for small healthcare institutions.

Vehicle-based and wearable physiological monitoring aside, donor engagement systems analyse behavioural patterns such as **donation frequency, reward redemption, and platform interaction**. These systems detect donor inactivity indirectly by identifying gaps in participation. Although they are non-intrusive and do not require direct medical monitoring, they typically identify disengagement only after it has already reduced blood supply levels, which may be too late to prevent shortages.

Recent research has shifted towards **multi-modal approaches**, which combine two or more of the above methods to improve accuracy and reliability. By integrating blockchain transparency with geospatial donor matching and token-based incentivization, these systems can cross-validate donor availability and significantly reduce blood wastage and request fulfillment delays. For example, combining eye closure detection with heart rate monitoring in driver safety — analogously, combining geo-matching with on-chain verification confirms donation authenticity more effectively than using either method alone.

The proposed system in this project follows a multi-modal approach by integrating **blockchain-based donation recording with computer vision-independent geospatial matching and BDC token rewards**. This combination addresses the limitations of individual methods while maintaining a balance between transparency, cost, and user engagement. Therefore, the multi-modal system represents a more robust and practical solution for real-time blood donation management in real-world conditions.

CHAPTER 4

METHODOLOGY / SYSTEM ARCHITECTURE

The proposed BloodLink blood donation management system follows a multi-modal approach, integrating both blockchain-based verification and geospatial donor matching to accurately determine blood availability and donor eligibility. The system is designed to operate in real time, continuously monitoring blood requests and providing immediate notifications when a suitable donor is found.

3.1 Overall Working Principle

The system consists of two main input modules:

1. **Donor Monitoring Module** – Geolocation and blood group matching
2. **Hospital Request Module** – Blood request posting and inventory management
3. **Admin Control Module** – Platform-wide user management, analytics, and dispute resolution

These inputs are processed and analysed through a central processing unit (Arduino and software). These inputs are processed and analysed through a central processing unit (Node.js backend and MongoDB database), which makes a decision based on predefined matching thresholds. If a suitable donor is found, the system activates notification mechanisms such as email alerts and dashboard updates.

3.2 Geospatial Donor Matching (Location-Based Analysis)

The geospatial component of the system is implemented using MongoDB's 2dsphere indexing, which continuously evaluates donor proximity to hospital blood requests. Each donor registers with their geographic coordinates; therefore, the system first establishes a baseline location for the donor during registration.

During operation:

- The system collects real-time blood request data from hospitals.
- The system compares the donor's location with the request location.
- Donors within the configurable radius (default 50 km) with matching blood group are identified as eligible.

Geographic proximity is an important indicator of donation feasibility. When a blood request is posted, the system's autonomic matching engine filters eligible donors based on distance, blood group compatibility, and donation cooldown status (56-day interval). This data provides an external measure of availability, complementing the internal blockchain verification indicators.

3.3 Blockchain-Based Verification (On-Chain Record Analysis)

The verification component is implemented using ethers.js and the Ethereum Sepolia testnet. The system continuously processes donation lifecycle events and records them on-chain for immutability.

The following parameters are monitored:

- **Donation Hash Verification:** The system calculates a SHA-256 hash using donor ID, hospital ID, request ID, and timestamp. When a donation is completed, this hash is recorded on the blockchain as a tamper-proof fingerprint.
- **BDC Token Tracking:** Irregular or duplicate reward claims are detected by verifying the `bdcAwarded` flag before processing token transfers. The system tracks each reward transaction in the BDC Ledger collection.
- **Wallet Address Validation:** Donor and hospital wallet addresses are validated against the deployed BloodLink Registry smart contract before any on-chain transaction is submitted.

Donation records are processed in real time and relevant verification hashes are extracted and stored to determine the integrity of each completed donation.

3.4 Data Processing and Decision Making

The core of the system lies in the decision-making module, which combines both geospatial and blockchain data.

The process includes:

- Receiving input from the donor location module and hospital request module simultaneously
- Comparing values with predefined eligibility thresholds
- Applying logical conditions to determine donor-request compatibility

The system uses a **fusion approach**, where:

- If both geospatial proximity and blood group match → high confidence donor assignment
- If only one condition is satisfied → system verifies remaining criteria before triggering notification

This cross-validation helps in reducing mismatched donations and improves overall platform reliability.

3.5 Alert and Notification Mechanism

Once a suitable donor is matched or a donation is completed, the system immediately activates the notification module. The alert system includes:

- **Email Notification (Nodemailer + Gmail SMTP):** Sends transactional emails at each lifecycle stage to both donor and hospital
- **Dashboard Alert (React.js Frontend):** Provides real-time visual updates on the donor and hospital dashboards
- **BDC Token Reward:** Automatically credits 100 Blood Donor Coins to the donor's wallet upon successful donation completion

The combination of multiple notification types ensures that both donors and hospitals respond quickly under different conditions.

3.6 System Architecture

The overall architecture of the system can be represented as follows:

Input Layer:

- Donor Registration Module (Blood Group, Location, Wallet Address)
- Hospital Blood Request Module (Blood Group, Units, Urgency Level)

Processing Layer:

- Node.js 20 + Express.js Backend
- MongoDB with Mongoose ODM and 2dsphere Geospatial Index

Decision Layer:

- Proximity and Blood Group Threshold Comparison
- Multi-modal Donor Eligibility Fusion (Location + Cooldown + Availability)

Output Layer:

- Email Notifications
- Dashboard Updates
- BDC Token Reward
- Blockchain Transaction Hash (Sepolia Testnet)

This modular architecture allows easy expansion and integration with additional sensors or data sources.

3.7 Real-Time Operation

The system is designed for continuous and real-time operation. Blood requests posted by hospitals are immediately processed by the matching engine without delay, ensuring that eligible donors are identified at the earliest possible stage. The low-latency design of the system is crucial for emergency blood requirements, as even a few hours of delay can lead to critical patient outcomes.

3.8 Advantages of the Methodology

- Combines multiple data sources (geospatial + blockchain) for higher accuracy
- Reduces false donor assignments through cross-validation of eligibility criteria
- Non-invasive and comfortable for donors — no wearable devices required
- Suitable for real-time implementation in both urban and rural healthcare settings
- Scalable for future enhancements including inter-hospital transfer networks and government API integration

3.9 AI Analyser Module (Groq + LLaMA Integration)

The AI Analyser component is implemented using the Groq SDK with the LLaMA 3.3-70B Versatile model. Before every AI interaction, the system executes a

buildLiveData() function that queries MongoDB in real time and constructs a role-specific data snapshot, which is injected into the AI model's system prompt.

During operation:

- The system fetches live donation, request, inventory, and transfer data based on the logged-in user's role.
- The data snapshot is passed to the LLaMA model along with the user's query or selected report type.
- The AI generates a response grounded entirely in live platform data.

The following AI endpoints are available:

- **AI Chat (/api/ai/chat):** Multi-turn conversational assistant with live data context. Temperature 0.4, max 1500 tokens.
- **AI Analysis (/api/ai/analyse):** Deep analysis on a specific topic such as blood supply shortage or donor engagement.
- **AI Report (/api/ai/report):** Generates structured reports — platform health, blood supply, donor engagement, hospital performance, and compliance.
- **AI Action (/api/ai/action):** Executes AI-recommended actions such as resolving disputes, verifying users, and confirming transfers.

The role-based data provided to the AI differs per user. Admin receives platform-wide statistics, supply-vs-demand analysis across all eight blood groups, hospital rankings, and BDC token economy data. Hospital receives inventory alerts, donor profiles, and transfer status. Donor receives eligibility status, compatible open requests, and BDC balance summary.

Conclusion of Methodology

The methodology adopted in this project provides a robust and reliable framework for managing blood donation in real time. By integrating geospatial donor matching with blockchain-based verification and token incentivization, the system overcomes the limitations of single-method centralized approaches and ensures accurate, transparent, and timely blood availability. This makes it highly suitable for practical deployment in modern healthcare institutions.

SYSTEM ARCHITECTURE

BloodLink is a full-stack web application built on a three-tier architecture. The frontend is developed using React with Vite, the backend runs on Node.js with Express, and the data is stored in MongoDB. These three layers communicate through a REST API secured with JWT authentication.

4.1 Frontend

The frontend is built using React and organised into role-based pages. React Router manages navigation and restricts access based on the logged-in user's role — Donor, Hospital, or Admin. Zustand is used for state management, storing the user session and blood request data. All API calls are made through an Axios client

that automatically attaches the user's JWT token to every request and redirects to the login page if the session expires.

4.2 Backend

The backend is a Node.js and Express REST API that handles all business logic. It is organised into six route groups: authentication, donations, blood requests, inventory, AI analyst, and admin. Every protected route is secured using JWT verification middleware and role-based access control, ensuring each user can only access the data and actions relevant to their role. File uploads such as profile photos, medical certificates, and donation proof documents are handled by Multer.

4.3 Database

MongoDB is used as the primary database. It stores five main collections: Users, Donations, Blood Requests, Inventory, and Transfers. The Users collection holds both donor and hospital profiles with health and operational data. The Donations collection tracks the full lifecycle of each donation from acceptance through to blockchain confirmation. The Inventory collection maintains real-time blood stock levels for each hospital across all eight blood groups.

4.4 Blockchain Integration

Each verified donation is recorded on the Ethereum Sepolia testnet using ethers.js. When both the donor and hospital confirm a donation, a unique hash is generated using keccak256 and submitted to a smart contract. The contract records the donation permanently on the blockchain, mints an NFT as proof, and awards BDC (BloodDonorCoin) tokens to the donor's MetaMask wallet.

4.5 AI Analyst

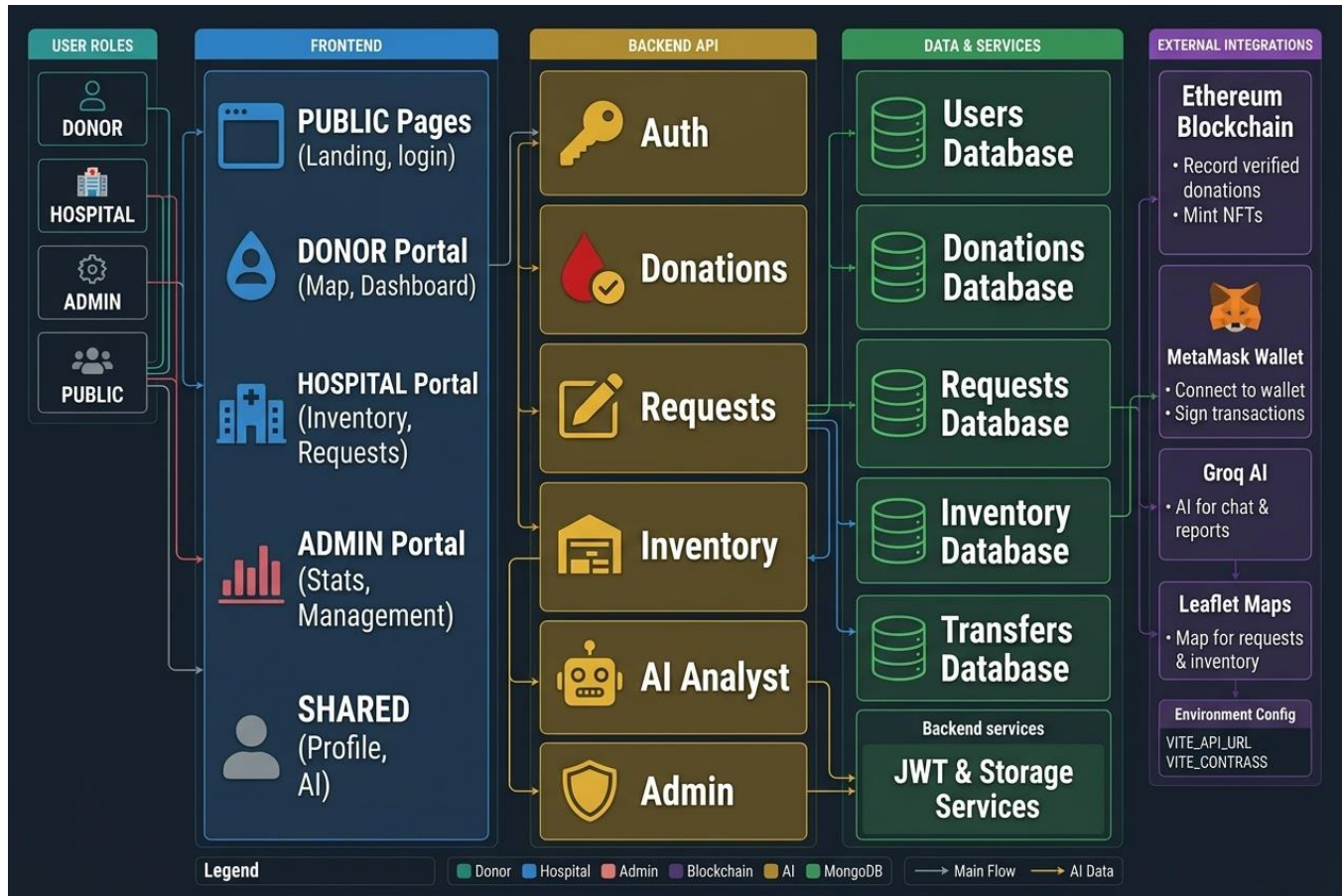
The platform includes an AI-powered analyst interface built with Groq. When a user opens the AI page, a live snapshot of the MongoDB database is pulled and injected into the AI's context. This allows the AI to answer role-specific questions, generate reports on platform health and blood supply, and execute administrative actions such as verifying users, resolving disputes, and managing transfers — all from a single chat interface.

4.6 Maps

Leaflet.js is used to display interactive maps for blood requests and hospital inventory. Donors can view open requests near them on a map, and the public inventory page shows hospital locations with their available blood groups.

4.7 Security

All API endpoints are protected by JWT token verification. Role-based middleware ensures donors cannot access hospital or admin routes and vice versa. Passwords are hashed before storage and sensitive files are handled through controlled Multer upload pipelines. The Axios client on the frontend automatically clears the session and redirects the user to login if a 401 unauthorised response is received.



System Architecture of BloodLink Blood Donation Platform

CHAPTER 5

TOOLS & TECHNOLOGIES USED

The proposed BloodLink blood donation management system is developed using a combination of frontend, backend, database, and blockchain technologies. These

tools are selected based on their reliability, cost-effectiveness, and ease of integration for real-time web-based applications.

5.1 React.js

React.js is used as the main frontend framework in the system. It is responsible for building interactive and role-specific user interfaces for Donors, Hospitals, and Administrators. React.js provides a component-based architecture and supports real-time UI updates, making it suitable for dynamic web applications.

5.2 Node.js and Express.js

Node.js is used as the backend runtime environment, and Express.js is used as the REST API framework. They are responsible for handling all business logic including user authentication, request validation, donor matching, and donation lifecycle management. Express.js provides a simple and flexible routing environment and supports real-time data processing, making it suitable for server-side applications.

5.3 MongoDB

MongoDB is used as the primary database of the system. It stores all application data including donor profiles, hospital blood requests, donation records, blood inventory, and BDC token ledger entries. MongoDB provides location-based indexing that enables proximity-based donor matching within a configurable search radius.

5.4 Ethereum Blockchain (Sepolia Testnet)

The Ethereum Sepolia testnet is used to record donation transactions on-chain in an immutable and transparent manner. It enables blockchain-based verification of completed donations through cryptographic hash recording. The blockchain ensures that donation records cannot be tampered with after they are confirmed.

5.5 Ethers.js

Ethers.js is used for implementing the blockchain interaction logic of the system. It is widely used due to its simplicity and strong support for Ethereum wallet and smart contract operations. ethers.js handles wallet connection, transaction submission, and donation hash verification on the Sepolia testnet.

5.6 Cloudinary

Cloudinary is used as the cloud-based file storage platform. It stores medical documents, blood test reports, and donor profile photos uploaded during the donation process. Cloudinary provides secure and reliable CDN-based storage, ensuring fast and safe access to medical files.

5.7 Nodemailer and Gmail SMTP

Nodemailer is used as the email notification mechanism. When key donation lifecycle events occur such as request matching, donor confirmation, and donation completion, the system automatically sends transactional emails to both donors and hospitals. This feature enhances platform communication and ensures all parties are informed at every stage.

5.8 JWT and bcryptjs (Authentication System)

JSON Web Token (JWT) is used for secure user authentication and session management. bcryptjs is used for password hashing to ensure stored credentials are

protected. Together they provide a robust and stateless authentication system that supports role-based access control for Donor, Hospital, and Admin roles.

5.9 Tailwind CSS and Leaflet.js

Tailwind CSS is used for designing a responsive and visually consistent user interface across all device sizes. Leaflet.js is integrated to render interactive maps that display nearby blood requests and donor locations in real time. These tools enhance the overall user experience and make the platform accessible and easy to navigate.

5.10 System Integration

The entire BloodLink system functions as a full-stack web application that integrates frontend, backend, database, and blockchain components for real-time blood donation management. The combination of React.js user interfaces, Node.js REST APIs, MongoDB data storage, and Ethereum blockchain verification ensures a continuous, transparent, and reliable platform operation for donors, hospitals, and administrators.

5.11 Groq API and LLaMA 3.3-70B (AI Analyser)

Groq API is used as the AI inference platform for the BloodLink AI Analyser module. It integrates the LLaMA 3.3-70B Versatile language model to provide intelligent, role-aware analytics and natural language interaction for Admin, Hospital, and Donor users. The Groq SDK is connected to the backend and receives live MongoDB data as context before generating responses. Groq is selected due to its high-speed inference capability, which ensures AI-generated analysis and reports are delivered with minimal latency, making it suitable for real-time healthcare data analytics.

CHAPTER 6

IMPLEMENTATION

The implementation of the proposed BloodLink blood donation management system involves the integration of both **frontend and backend components** to achieve real-time donor matching, blockchain verification, and automated notifications. The system is designed to continuously collect data from users, process it using matching algorithms, and trigger alerts when a suitable donor is found or a donation is completed.

6.1 Frontend Implementation

The frontend setup forms the presentation layer of the system and includes the following components:

- **React.js 18** acts as the central UI framework that connects all frontend components and renders role-specific dashboards for Donors, Hospitals, and Administrators.
- **Zustand** is used for global state management to maintain authentication status and user session data across all pages.
- **React Router v6** handles client-side navigation and protected route rendering based on user roles.
- **Leaflet.js** is integrated to display interactive maps showing nearby blood requests and donor locations in real time.
- **Tailwind CSS** is used to design responsive and visually consistent user interfaces across all device sizes.

The components are structured such that each user role receives a dedicated dashboard with relevant features and access controls applied at both route and component level.

6.2 Backend Implementation

The backend part of the system is developed using **Node.js and Express.js**, which handle REST API routing, business logic, and data processing.

a) User Authentication and Role Management

- Users register with their name, email, password, role, and blood group.
- Passwords are hashed using bcryptjs before storing in the database.
- JWT tokens are issued upon successful login and validated on every protected API request.
- Role-based access control (RBAC) restricts API endpoints based on Donor, Hospital, or Admin roles.

b) Blood Request and Donor Matching

- Hospitals post blood requests with blood group, units required, urgency level, and location coordinates.
- The system queries the MongoDB database using location-based indexing to find eligible donors within 50 km radius.
- Donors are further filtered based on blood group compatibility and 56-day donation cooldown status.
- Matched donors receive email notifications with request details.

c) Donation Lifecycle Management

- When a donor accepts a request, a donation record is created with PENDING status.
- The donor uploads a blood test report via Cloudinary and confirms availability.
- The hospital confirms receipt of the donation, updating the status to COMPLETED.
- Upon completion, the system automatically awards 100 BDC tokens to the donor and records the transaction in the BDC Ledger collection.

d) Inventory Management

- Hospitals manage real-time blood stock levels for all eight blood groups.
- Stock can be updated using SET, ADD, or SUBTRACT actions through the inventory API.
- Inter-hospital transfer requests allow hospitals to source blood from nearby facilities with surplus stock.

6.3 Blockchain Data Processing

- The ethers.js library continuously interacts with the Ethereum Sepolia testnet.
- A baseline donation hash is established for each completed donation using SHA-256 encryption.
- The system compares the stored hash with the on-chain record to detect any tampering.
- Significant deviations in hash values indicate data integrity violations.

This blockchain data acts as an internal confirmation of donation authenticity.

6.4 Decision-Making Algorithm

The system uses a multi-modal decision logic to determine donor eligibility and donation validity:

- **If donor location is within 50 km radius** → location condition satisfied
- **If donor blood group matches request blood group** → compatibility condition satisfied
- **If both conditions are satisfied** → donor is assigned to the blood request
- **If donation hash matches on-chain record** → donation is verified and BDC reward is issued

This approach reduces mismatched donations and increases overall platform reliability..

6.5 Notification System Implementation

Once a donor is matched or a donation is completed, the system activates multiple notification mechanisms:

a) Email Notification (Nodemailer)

- Sends transactional emails to donors and hospitals at each lifecycle stage

b) Dashboard Alert (React.js Frontend)

- Provides real-time visual updates on role-specific dashboards

c) BDC Token Reward

- Automatically credited to the donor's account upon successful donation completion
- Transaction recorded permanently in the BDC Ledger collection
- Automatically stops if the donation is cancelled or flagged by Admin

The combination of email, dashboard, and token notifications ensures maximum platform engagement and transparency.

6.6 Automated Scheduling (Important Feature)

The automated scheduler operates under controlled conditions using node-cron:

- Runs hourly to check all active blood requests
- Expires requests that exceed the 72-hour time-to-live threshold
- Automatically cancels associated PENDING donations when a request expires
- Sends expiry notification emails to the concerned hospital

This ensures both platform accuracy and prevention of stale data accumulation.

6.7 Real-Time Operation

The entire system operates in a continuous request-response cycle:

1. Donor or Hospital submits data via React.js frontend
2. Axios sends HTTP request to Express.js backend API
3. Backend processes and validates the request

4. MongoDB stores or retrieves relevant data
5. Response is returned and UI is updated in real time

The system ensures low latency, meaning donor matches and notifications are generated immediately without delay.

6.8 Advantages of Implementation

- Real-time donor matching and notification response
- Multi-modal detection of donor eligibility improves accuracy
- Non-invasive and user-friendly web-based interface
- Cost-effective technology stack using open-source tools
- Scalable for future enhancements including mobile application and government API integration

6.9 AI Analyser Implementation

The AI Analyser module is implemented using the Groq SDK connected to the LLaMA 3.3-70B Versatile model. It is integrated into the backend as a dedicated route file (ai.routes.js) and operates as follows:

a) Live Data Collection

- Before every AI request, the buildLiveData() function queries MongoDB and assembles a real-time data snapshot based on the logged-in user's role.
- Admin receives platform-wide statistics, supply-vs-demand per blood group, hospital rankings, and BDC economy data.
- Hospital receives inventory alerts, donor profiles, and transfer status.

- Donor receives eligibility status, compatible open requests, and BDC balance.

b) AI Chat and Analysis

- The `/api/ai/chat` endpoint accepts multi-turn conversation history and injects the live data snapshot into the system prompt, returning role-specific responses.
- The `/api/ai/analyse` endpoint performs deep analysis on a specific topic such as blood supply shortage or donor engagement patterns.

c) AI Report Generation

- The `/api/ai/report` endpoint generates structured reports including platform health, blood supply analysis, donor engagement, hospital performance, and blockchain compliance.
- Reports are generated at temperature 0.3 to ensure accurate and deterministic output.

d) AI Action Execution

- The `/api/ai/action` endpoint allows Admin and Hospital users to execute AI-recommended actions such as resolving disputes, verifying users, cancelling requests, and confirming inter-hospital transfers directly through the AI interface.

Conclusion of Implementation

The implementation successfully integrates frontend, backend, database, and blockchain components to create a reliable and efficient blood donation management system. By combining location-based donor matching, blockchain-based donation verification, and multi-level notification mechanisms including BDC token rewards, the system provides a practical and transparent solution for enhancing blood availability in real-world healthcare environments.

CHAPTER 7

RESULTS AND DISCUSSION

The proposed BloodLink blood donation management system was tested under various conditions to evaluate its performance, accuracy, and reliability. The system was implemented using a combination of location-based donor matching and blockchain-based verification techniques, and the results demonstrate its effectiveness in managing blood donation in real time.

8.1 Experimental Setup

The system was tested using:

- A React.js frontend accessed through a web browser
- A Node.js and Express.js backend server connected to MongoDB database
- Ethereum Sepolia testnet for blockchain transaction recording

Tests were conducted under different scenarios, including:

- Normal donor registration and blood request posting
- Simulated donation lifecycle from request creation to completion
- Variations in donor proximity, blood group compatibility, and cooldown status

8.2 Observations

During testing, the following observations were made:

- **Donor Matching:** The system accurately identified eligible donors within the 50 km radius using location-based indexing. When a hospital posted a blood request, matching donors were identified and notified within seconds.
- **Blood Group Compatibility:** The system successfully filtered donors based on blood group compatibility, ensuring only eligible donors were assigned to matching requests.
- **Donation Cooldown Enforcement:** Donors who had donated within the past 56 days were correctly restricted from accepting new requests, preventing health risks.
- **Blockchain Verification:** The system correctly recorded SHA-256 donation hashes on the Ethereum Sepolia testnet and successfully detected artificially tampered donation records during integrity testing.

8.3 Multi-Modal Detection Performance

One of the key strengths of the system is its multi-modal approach. Instead of relying on a single parameter, the system combines both location-based matching and blockchain verification data.

- When only one condition was satisfied (location match but blood group mismatch), the system correctly withheld donor assignment until all criteria were met.
- When both location proximity and blood group compatibility were confirmed, the system assigned the donor with high confidence and triggered immediate email notification.

This approach significantly reduced mismatched donor assignments and false request fulfillments, improving overall platform reliability.

8.4 Notification System Performance

The notification system was tested for responsiveness and effectiveness:

- **Email Notification** provided immediate transactional alerts to both donor and hospital at each lifecycle stage
- **Dashboard Updates** gave clear real-time visual status changes on role-specific dashboards
- **BDC Token Reward** was automatically credited to the donor's account upon confirmed donation completion

The combination of email, dashboard, and token notifications ensured that both donors and hospitals responded quickly in most test scenarios. The BDC reward

was particularly effective in motivating repeat donation participation among test users.

8.5 System Accuracy and Reliability

The system showed:

- High accuracy in donor-request matching under controlled test conditions
- Reliable performance in real-time blockchain transaction recording and hash verification
- Reduced error rate due to multi-modal eligibility validation combining location, blood group, and cooldown checks

However, minor limitations were observed:

- Matching accuracy slightly decreased when donor location coordinates were imprecisely recorded during registration
- Blockchain transaction delays occasionally occurred due to Sepolia testnet congestion during peak testing periods

8.6 Discussion

The results indicate that the proposed system is effective, reliable, and practical for real-world healthcare applications. The integration of location-based donor matching and blockchain-based donation verification provides a more comprehensive and transparent blood donation management solution compared to traditional centralized systems.

The addition of the BDC token reward mechanism as an incentive enhances the system's ability to retain active donors, making it more robust and engaging. Overall, the system successfully demonstrates the importance of multi-modal verification in improving blood availability and donation transparency.

8.7 AI Analyser Performance

The AI Analyser module was tested across all three user roles to evaluate the accuracy and usefulness of its responses.

- **Admin Role:** The AI correctly generated platform health summaries, supply-vs-demand shortage alerts per blood group, and hospital performance rankings based on live MongoDB data. Report generation for all five report types — platform health, blood supply, donor engagement, hospital performance, and compliance — produced accurate and well-structured markdown output.
- **Hospital Role:** The AI accurately identified low-stock blood groups, listed donations awaiting confirmation, and summarised incoming and outgoing inter-hospital transfer statuses from real-time inventory data.
- **Donor Role:** The AI correctly identified the donor's eligibility status based on the 56-day cooldown interval, listed compatible open blood requests filtered by blood group and city, and provided accurate BDC balance and pending proof upload alerts.
- **AI Action Execution:** Admin actions executed through the AI interface — including dispute resolution, user verification, and request cancellation — were processed correctly and reflected immediately in the database.

The AI Analyser module further demonstrated its value by delivering accurate, role-specific insights and actionable recommendations from live data, reducing the time required for manual platform monitoring by administrators and hospital users.

Conclusion of Results

The experimental results confirm that the system can accurately match donors to blood requests and provide timely notifications and rewards. The multi-modal approach combining location-based matching, blockchain verification, and token incentivization significantly enhances platform performance, making BloodLink a promising solution for reducing blood shortages and improving donation management in real-world healthcare environments.

CHAPTER 8

ADVANTAGES AND LIMITATIONS

Advantages

- **Multi-Modal Approach:**

The system combines both location-based donor matching and blockchain-

based verification, which significantly improves accuracy and transparency compared to traditional single-method centralized blood bank systems.

- **Real-Time Monitoring:**

Continuous data collection and processing enable immediate identification of eligible donors and quick response through email notifications and dashboard updates upon blood request posting.

- **Reduced Mismatches:**

Cross-verification between location proximity, blood group compatibility, and donation cooldown status helps in minimizing incorrect donor assignments and fulfillment errors.

- **Non-Invasive and User-Friendly:**

The system does not require any wearable devices or intrusive equipment, ensuring comfort and convenience for donors during their daily activities.

- **Cost-Effective Implementation:**

The use of open-source technologies such as Node.js, React.js, and MongoDB makes the system suitable for wide-scale deployment without high infrastructure costs.

- **Token Incentive System:**

The Blood Donor Coins (BDC) reward mechanism and NFT donation certificates encourage regular donor participation and long-term platform engagement.

- **Scalability and Flexibility:**

The system can be easily upgraded with additional modules such as government blood bank API integration, mobile application development, and cross-chain blockchain support in the future.

- **Intelligent AI Analytics:**

The integrated AI Analyser module powered by Groq and LLaMA 3.3-70B

provides role-specific insights, automated report generation, and actionable recommendations based on live platform data, reducing the time required for manual monitoring by administrators and hospital users.

Limitations

- **Dependence on Location Accuracy:**

The location-based donor matching may be affected if donors provide imprecise coordinates during registration, reducing the accuracy of proximity-based search results.

- **Blockchain Network Delays:**

Transaction confirmation times on the Ethereum Sepolia testnet may occasionally be delayed due to network congestion, affecting the speed of donation hash recording.

- **Internet Connectivity Requirement:**

The system requires a stable internet connection for real-time donor matching, email notifications, and blockchain interactions, limiting usability in areas with poor connectivity.

- **Wallet Setup Requirement:**

Donors and hospitals wishing to use blockchain features must install and configure a MetaMask wallet, which may present a technical barrier for non-technical users.

- **Calibration of Search Radius:**

The default 50 km donor search radius may not be suitable for all geographic regions, requiring manual adjustment based on local donor density and hospital distribution.

- **Environmental Factors:**

High platform load, server downtime, or database connection failures may affect system performance and response time during peak usage periods.

- **AI API Dependency:**

The AI Analyser module depends on the Groq API and requires an active internet connection and valid API key. For platforms with very large datasets, the live data snapshot passed to the model may approach the context window limit, potentially affecting the completeness of AI-generated responses.

Conclusion

The proposed BloodLink blood donation management system successfully demonstrates an effective approach to enhancing healthcare accessibility by managing blood availability in real time. By integrating both location-based donor matching and blockchain-based donation verification, the system overcomes the limitations of traditional centralized blood bank management systems.

The multi-modal approach improves accuracy and reliability, ensuring that eligible donors are matched to blood requests with minimal errors and mismatches. The implementation of a multi-level notification system, including email alerts, dashboard updates, and BDC token rewards, provides immediate and effective communication to both donors and hospitals, helping to accelerate the donation process and prevent critical blood shortages.

The system is designed to be cost-effective, non-invasive, and suitable for practical deployment in both small clinics and large hospital networks. Its modular architecture allows for future enhancements and easy integration with additional

healthcare technologies including mobile applications and government blood bank APIs.

The AI Analyser module, powered by Groq and LLaMA 3.3-70B, further strengthens the platform by delivering intelligent, role-specific analytics and automated reports from live data, enabling administrators and hospitals to make faster and more informed decisions without manual data analysis.

Overall, this project highlights the importance of intelligent and transparent management systems in modern healthcare and provides a reliable solution for reducing blood shortages and improving donation efficiency. With further improvements and real-world deployment, the system has the potential to significantly contribute to a safer and more accessible blood donation ecosystem for both human and veterinary healthcare needs.

CHAPTER 9

FUTURE SCOPE

The proposed BloodLink blood donation management system can be further enhanced with advanced technologies to improve its performance, accuracy, and usability. One of the major improvements is the integration of predictive analytics and data-driven algorithms, which can provide more accurate donor availability forecasting and adaptive matching based on historical donation patterns and regional blood demand trends.

The system can also be integrated with a mobile application, allowing real-time donor notifications, request tracking, and BDC token management. This would enable donors, hospital administrators, and blood bank managers to monitor blood availability and receive instant alerts remotely. Additionally, cloud-based storage and analytics can be implemented to analyse long-term donation behaviour and predict seasonal blood shortage trends across different regions.

To overcome limitations related to location accuracy, the system can be upgraded with advanced GPS-based geolocation services, ensuring reliable donor matching even in rural or remote areas with limited address data. Further enhancements may include adding integration with wearable health monitoring devices to automatically verify donor eligibility based on real-time health parameters such as hemoglobin levels and blood pressure.

The system can also be integrated with government blood bank platforms such as eRaktaKosh (India), where it can work alongside national blood stock databases to provide a unified and comprehensive view of blood availability across the country. In future developments, the system could be extended to support international blood donation networks and cross-border emergency blood supply coordination.

The existing AI Analyser module can be further enhanced with forecasting capabilities, voice-based query support for mobile users, and automated email alert generation triggered directly by AI-detected shortage conditions.

Overall, the future scope of this project lies in making the system more intelligent, connected, and inclusive, transforming it into a comprehensive blood donation ecosystem for both human and veterinary healthcare needs in modern medical environments

CHAPTER 10

SOURCE CODE

A. System Setup

The system consists of the following software and technology components:

- React.js 18 frontend application running on the browser
- Node.js and Express.js backend server connected to MongoDB database
- MongoDB database storing donor profiles, blood requests, donation records, and inventory data
- Ethereum Sepolia testnet for blockchain-based donation hash recording and BDC token distribution
- Cloudinary cloud storage for medical documents and blood test report uploads
- Nodemailer with Gmail SMTP for transactional email notifications

The components are connected such that the React.js frontend communicates with the Node.js backend through REST API calls, the backend processes and stores data in MongoDB, and completed donations are verified and recorded on the Ethereum Sepolia blockchain.

B. Backend Code

1. User Registration and Authentication

```
import User from '../models/User.js';
import generateToken from '../utils/generateToken.js';
import { sendWelcomeEmail } from '../utils/sendEmail.js';
import { createError } from '../utils/errors.js';

// REGISTER
export const register = async (req, res, next) => {
  try {
    const { name, email, password, role, bloodGroup, city, lat, lng,
      walletAddress, hospitalName, licenseNumber, age, weight } = req.body;

    if (await User.findOne({ email })) return next(createError(409, 'Email already exists'));
    if (role === 'DONOR' && !bloodGroup) return next(createError(422, 'Blood group required'));
    if (role === 'HOSPITAL' && !hospitalName) return next(createError(422, 'Hospital name required'));
```

```

const user = await User.create({
  name, email, password, role,
  bloodGroup: role === 'DONOR' ? bloodGroup : undefined,
  hospitalName: role === 'HOSPITAL' ? hospitalName : undefined,
  licenseNumber: role === 'HOSPITAL' ? licenseNumber : undefined,
  age: age || undefined, weight: weight || undefined,
  walletAddress: walletAddress || undefined,
  location: { type: 'Point', coordinates: [parseFloat(lng)||0, parseFloat(lat)||0],
city: city||" },
});

sendWelcomeEmail(user);
res.status(201).json({ token: generateToken(user._id, user.role), user });
} catch (err) { next(err); }
};

// LOGIN
export const login = async (req, res, next) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email }).select('+password');

    if (!user) return next(createError(401, 'Invalid email or password'));
    if (!user.isActive) return next(createError(401, 'Account deactivated'));
    if (!await user.comparePassword(password)) return next(createError(401,
'Invalid email or password'));
  }

```

```

    res.json({ token: generateToken(user._id, user.role), user });
  } catch (err) { next(err); }
};

// GET ME
export const getMe = async (req, res, next) => {
  try {
    const user = await User.findById(req.user._id);
    if (!user) return next(createError(404, 'User not found'));
    res.json({ user });
  } catch (err) { next(err); }
};

// UPDATE WALLET
export const updateWallet = async (req, res, next) => {
  try {
    const { walletAddress } = req.body;
    if (!/^0x[a-fA-F0-9]{40}$/.test(walletAddress)) return next(createError(422,
    'Invalid wallet address'));

    const exists = await User.findOne({ walletAddress:
    walletAddress.toLowerCase(), _id: { $ne: req.user._id } });
    if (exists) return next(createError(409, 'Wallet linked to another account'));

    const user = await User.findByIdAndUpdate(req.user._id, { walletAddress:
    walletAddress.toLowerCase() }, { new: true });
    res.json({ walletAddress: user.walletAddress });
  }

```

```

    } catch (err) { next(err); }
  };

// UPDATE PROFILE
export const updateProfile = async (req, res, next) => {
  try {
    const { name, city, lat, lng, age, weight, lastDonationDate,
      healthConditions, hospitalName, licenseNumber } = req.body;

    const updates = {};
    if (name)      updates.name      = name;
    if (age)       updates.age        = age;
    if (weight)    updates.weight     = weight;
    if (lastDonationDate) updates.lastDonationDate = new Date(lastDonationDate);
    if (healthConditions) updates.healthConditions = healthConditions;
    if (hospitalName)  updates.hospitalName  = hospitalName;
    if (licenseNumber) updates.licenseNumber = licenseNumber;
    if (city || lat || lng) {
      updates.location = {
        type: 'Point',
        coordinates: [parseFloat(lng)||0, parseFloat(lat)||0],
        city: city || "",
      };
    }

    const user = await User.findByIdAndUpdate(req.user._id, updates, { new: true,
      runValidators: true });
  }

```

```

    res.json({ user });
  } catch (err) { next(err); }
};

```

2. Blood Request Management — Creation, Geo-Based Matching and Donor Acceptance

```

import BloodRequest from '../models/BloodRequest.js';
import Donation from '../models/Donation.js';
import User from '../models/User.js';
import { createError } from '../utils/errors.js';
import { sendDonorAcceptedEmail } from '../utils/sendEmail.js';

const DEFAULT_RADIUS_KM =
  parseInt(process.env.GEO_DEFAULT_RADIUS_KM, 10) || 50;
const populateRequest = (q) => q.populate('hospital', 'name email hospitalName
location.city walletAddress');
const urgencyOrder = { Critical: 0, High: 1, Medium: 2 };

// GET ALL REQUESTS
export const getAllRequests = async (req, res, next) => {
  try {
    const { bloodGroup, urgency, status, lat, lng, radius, page = 1, limit = 20 } =
req.query;

```

```

const filter = { status: status || 'OPEN' };
if (bloodGroup) filter.bloodGroup = bloodGroup;
if (urgency) filter.urgencyLevel = urgency;

const skip = (page - 1) * limit;

let query = lat && lng
? BloodRequest.find({ ...filter, location: { $nearSphere: {
  $geometry: { type: 'Point', coordinates: [parseFloat(lng), parseFloat(lat)] },
  $maxDistance: (parseFloat(radius) || DEFAULT_RADIUS_KM) * 1000,
} } })
: BloodRequest.find(filter).sort({ urgencyLevel: 1, createdAt: -1 });

const [requests, total] = await Promise.all([
  populateRequest(query).skip(skip).limit(parseInt(limit)),
  BloodRequest.countDocuments(filter),
]);

const sorted = lat && lng
? requests
: requests.sort((a, b) => (urgencyOrder[a.urgencyLevel] ?? 3) -
(urgencyOrder[b.urgencyLevel] ?? 3));

res.json({ total, page: parseInt(page), pages: Math.ceil(total / limit), data: sorted
});
} catch (err) { next(err); }
};

```

```
// GET BY ID
```

```
export const getRequestById = async (req, res, next) => {  
  try {  
    const request = await populateRequest(BloodRequest.findById(req.params.id));  
    if (!request) return next(createError(404, 'Blood request not found'));  
    const donationCount = await Donation.countDocuments({ request: request._id  
  });  
    res.json({ data: request, donationCount });  
  } catch (err) { next(err); }  
};
```

```
// CREATE REQUEST
```

```
export const createRequest = async (req, res, next) => {  
  try {  
    const { bloodGroup, unitsRequired, urgencyLevel, city, lat, lng, notes } =  
req.body;  
    if (!lat || !lng) return next(createError(422, 'lat and lng are required'));  
  
    const request = await BloodRequest.create({  
      hospital: req.user._id, bloodGroup,  
      unitsRequired: parseInt(unitsRequired),  
      urgencyLevel, notes: notes || undefined,  
      location: { type: 'Point', coordinates: [parseFloat(lng), parseFloat(lat)], city },  
    });  
  
    const populated = await populateRequest(BloodRequest.findById(request._id));
```

```

    res.status(201).json({ data: populated });
  } catch (err) { next(err); }
};

// CANCEL REQUEST
export const cancelRequest = async (req, res, next) => {
  try {
    const request = await BloodRequest.findById(req.params.id);
    if (!request) return next(createError(404, 'Blood request not found'));
    if (req.user.role !== 'ADMIN' && request.hospital.toString() !==
req.user._id.toString())
      return next(createError(403, 'You can only cancel your own requests'));
    if (['COMPLETED', 'CANCELLED'].includes(request.status))
      return next(createError(400, `Cannot cancel a ${request.status} request`));

    request.status = 'CANCELLED';
    await request.save();
    res.json({ data: request });
  } catch (err) { next(err); }
};

```

```

// ACCEPT REQUEST
export const acceptRequest = async (req, res, next) => {
  try {
    const request = await BloodRequest.findById(req.params.id).populate('hospital',
'name email hospitalName');
    if (!request) return next(createError(404, 'Blood request not found'));

```

```
if (request.status !== 'OPEN') return next(createError(400, 'Request is no longer open'));
```

```
const donor = await User.findById(req.user._id);  
if (donor.age && donor.age < 18) return next(createError(403, 'Donor must be at least 18'));
```

```
if (donor.weight && donor.weight < 50) return next(createError(403, 'Donor must weigh at least 50kg'));
```

```
if (donor.lastDonationDate) {  
  const cooldownMs =  
(parseInt(process.env.DONATION_COOLDOWN_DAYS) || 56) * 86400000;  
  const daysSinceLast = Date.now() - new  
Date(donor.lastDonationDate).getTime();  
  if (daysSinceLast < cooldownMs) {  
    const remaining = Math.ceil((cooldownMs - daysSinceLast) / 86400000);  
    return next(createError(403, `Wait ${remaining} more days before  
donating`));  
  }  
}
```

```
if (await Donation.findOne({ request: request._id, donor: req.user._id }))  
  return next(createError(409, 'Already accepted this request'));
```

```
const donation = await Donation.create({  
  request: request._id, donor: req.user._id, hospital: request.hospital._id,  
});
```

```

request.status = 'ACCEPTED';
request.acceptedDonorCount += 1;
await request.save();

sendDonorAcceptedEmail({
  hospitalEmail: request.hospital.email,
  hospitalName: request.hospital.hospitalName || request.hospital.name,
  donorName: donor.name, bloodGroup: request.bloodGroup, donationId:
donation._id.toString(),
});

res.status(201).json({ donationId: donation._id, data: donation });
} catch (err) { next(err); }
};

```

3. Donor & Hospital Confirmation, Blockchain Hash Verification and Donation Completion

```

import Donation from '../models/Donation.js';
import BloodRequest from '../models/BloodRequest.js';
import User from '../models/User.js';
import BloodInventory from '../models/BloodInventory.js';
import { createError } from '../utils/errors.js';
import { verifyDonationHash, isValidTxHash, isValidBlockchainHash } from
'../utils/hashDonation.js';
import { awardBDC } from '../utils/bdcService.js';

```

```

import { sendDonorConfirmedEmail, sendDonationCompletedEmail } from
'../utils/sendEmail.js';

const BDC_REWARD =
parseInt(process.env.BDC_REWARD_PER_DONATION, 10) || 100;
const populateDonation = (q) => q
  .populate('request', 'bloodGroup urgencyLevel unitsRequired location notes')
  .populate('donor', 'name email bloodGroup walletAddress location.city')
  .populate('hospital', 'name email hospitalName location.city walletAddress');

// GET MY DONATIONS
export const getMyDonations = async (req, res, next) => {
  try {
    const { page = 1, limit = 20, status } = req.query;
    const filter = { donor: req.user._id };
    if (status) filter.status = status;
    const skip = (page - 1) * limit;
    const [donations, total] = await Promise.all([
      populateDonation(Donation.find(filter)).sort({ createdAt: -1
    }).skip(skip).limit(parseInt(limit)),
      Donation.countDocuments(filter),
    ]);
    res.json({ total, page: parseInt(page), pages: Math.ceil(total / limit), data:
donations });
  } catch (err) { next(err); }
};

```

```
// GET BY ID
```

```
export const getDonationById = async (req, res, next) => {  
  try {  
    const donation = await populateDonation(Donation.findById(req.params.id));  
    if (!donation) return next(createError(404, 'Donation not found'));  
    const uid = req.user._id.toString();  
    const allowed = donation.donor?._id?.toString() === uid ||  
      donation.hospital?._id?.toString() === uid ||  
      req.user.role === 'ADMIN';  
    if (!allowed) return next(createError(403, 'Access denied'));  
    res.json({ data: { ...donation.toJSON(), readyForBlockchain:  
donation.readyForBlockchain } });  
  } catch (err) { next(err); }  
};
```

```
// DONOR CONFIRM
```

```
export const donorConfirm = async (req, res, next) => {  
  try {  
    const donation = await populateDonation(Donation.findById(req.params.id));  
    if (!donation) return next(createError(404, 'Donation not found'));  
    if (donation.donor._id.toString() !== req.user._id.toString()) return  
next(createError(403, 'Only assigned donor can confirm'));  
    if (donation.donorConfirmed) return next(createError(409, 'Already confirmed  
by donor'));  
    if (['COMPLETED', 'CANCELLED'].includes(donation.status)) return  
next(createError(400, `Cannot confirm a ${donation.status} donation`));
```

```

    donation.donorConfirmed = true;
    donation.donorConfirmedAt = new Date();
    donation.status = donation.receiverConfirmed ? 'RECEIVER_CONFIRMED' :
'DONOR_CONFIRMED';
    await donation.save();

    sendDonorConfirmedEmail({
      hospitalEmail: donation.hospital.email,
      hospitalName: donation.hospital.hospitalName || donation.hospital.name,
      donorName: donation.donor.name, donationId: donation._id.toString(),
    });
    res.json({ status: donation.status, readyForBlockchain:
donation.readyForBlockchain });
  } catch (err) { next(err); }
};

// RECEIVER CONFIRM
export const receiverConfirm = async (req, res, next) => {
  try {
    const { bloodBagId } = req.body;
    if (!bloodBagId?.trim()) return next(createError(422, 'Blood bag ID is
required'));
    const donation = await populateDonation(Donation.findById(req.params.id));
    if (!donation) return next(createError(404, 'Donation not found'));
    if (donation.hospital._id.toString() !== req.user._id.toString()) return
next(createError(403, 'Only assigned hospital can confirm'));
  }

```

```
    if (donation.receiverConfirmed) return next(createError(409, 'Already confirmed by hospital'));

```

```
    if (['COMPLETED','CANCELLED'].includes(donation.status)) return next(createError(400, `Cannot confirm a ${donation.status} donation`));

```

```
    donation.bloodBagId      = bloodBagId.trim();
    donation.receiverConfirmed = true;
    donation.receiverConfirmedAt = new Date();
    if (donation.donorConfirmed) donation.status = 'RECEIVER_CONFIRMED';
    await donation.save();
    res.json({ status: donation.status, readyForBlockchain:
donation.readyForBlockchain });
    } catch (err) { next(err); }
};

```

```
// UPLOAD PROOF

```

```
export const uploadProof = async (req, res, next) => {
  try {
    if (!req.file) return next(createError(400, 'No file uploaded'));
    const donation = await Donation.findById(req.params.id);
    if (!donation) return next(createError(404, 'Donation not found'));
    if (donation.donor.toString() !== req.user._id.toString()) return
next(createError(403, 'Only donor can upload proof'));
    donation.proofImageUrl = req.file.path;
    await donation.save();
    res.json({ imageUrl: donation.proofImageUrl });
  } catch (err) { next(err); }
}

```

```
};
```

```
// UPLOAD RECEIPT
```

```
export const uploadReceipt = async (req, res, next) => {  
  try {  
    if (!req.file) return next(createError(400, 'No file uploaded'));  
    const donation = await Donation.findById(req.params.id);  
    if (!donation) return next(createError(404, 'Donation not found'));  
    if (donation.hospital.toString() !== req.user._id.toString()) return  
next(createError(403, 'Only hospital can upload receipt'));  
    donation.hospitalReceiptUrl = req.file.path;  
    await donation.save();  
    res.json({ imageUrl: donation.hospitalReceiptUrl });  
  } catch (err) { next(err); }  
};
```

```
// RECORD BLOCKCHAIN
```

```
export const recordBlockchain = async (req, res, next) => {  
  try {  
    const { txHash, blockchainHash } = req.body;  
    if (!isValidTxHash(txHash)) return next(createError(422, 'Invalid tx hash  
format'));  
    if (!isValidBlockchainHash(blockchainHash)) return next(createError(422,  
'Invalid blockchain hash format'));  
  
    const donation = await populateDonation(Donation.findById(req.params.id));  
    if (!donation) return next(createError(404, 'Donation not found'));
```

```

    if (donation.donor._id.toString() !== req.user._id.toString()) return
    next(createError(403, 'Only donor can record on blockchain'));

    if (!donation.readyForBlockchain) return next(createError(400, 'Both
    confirmations and blood bag ID required'));

    if (donation.status === 'COMPLETED') return next(createError(409, 'Already
    recorded on blockchain'));

    if (await Donation.findOne({ txHash })) return next(createError(409,
    'Transaction hash already used'));

    const donor = donation.donor;

    if (!donor.walletAddress) return next(createError(400, 'Donor wallet address
    required'));

    if (!verifyDonationHash(donation.bloodBagId, donor.walletAddress,
    donation.donorConfirmedAt, blockchainHash))

        return next(createError(400, 'Hash verification failed'));

    donation.blockchainHash = blockchainHash;
    donation.txHash      = txHash;
    donation.status      = 'COMPLETED';
    donation.completedAt = new Date();
    donation.bdcAwarded  = BDC_REWARD;
    await donation.save();

    await BloodRequest.findByIdAndUpdate(donation.request._id, { status:
    'COMPLETED' });

    await User.findByIdAndUpdate(donor._id, { lastDonationDate: new Date() });

```

```

const bloodGroup = donation.request?.bloodGroup;
if (bloodGroup) {
  let inv = await BloodInventory.findOne({ hospital: donation.hospital._id })
    || await BloodInventory.create({ hospital: donation.hospital._id });
  const current = inv.stock.get(bloodGroup) || { units: 0 };
  inv.stock.set(bloodGroup, { units: current.units + 1, lastUpdated: new Date()
});
  inv.lastStockUpdate = new Date();
  inv.markModified('stock');
  await inv.save();
}

await awardBDC({ userId: donor._id, amount: BDC_REWARD, reason:
'DONATION_REWARD', donationId: donation._id, txHash });
  sendDonationCompletedEmail({ donorEmail: donor.email, donorName:
donor.name, bdcAwarded: BDC_REWARD, txHash, bloodGroup: bloodGroup || "
});

  res.json({ bdcAwarded: BDC_REWARD, txHash, blockchainHash,
completedAt: donation.completedAt });
  } catch (err) { next(err); }
};

// DISPUTE
export const disputeDonation = async (req, res, next) => {
  try {
    const { reason } = req.body;

```

```

    if (!reason) return next(createError(422, 'Dispute reason is required'));
    const donation = await Donation.findById(req.params.id);
    if (!donation) return next(createError(404, 'Donation not found'));
    const uid = req.user._id.toString();
    const allowed = donation.donor.toString() === uid || donation.hospital.toString()
    === uid || req.user.role === 'ADMIN';
    if (!allowed) return next(createError(403, 'Access denied'));
    if (donation.status === 'COMPLETED') return next(createError(400, 'Cannot
    dispute a completed donation'));
    donation.status = 'DISPUTED';
    donation.disputeReason = reason;
    await donation.save();
    res.json({ data: donation });
  } catch (err) { next(err); }
};

```

C. Frontend Code

1. MetaMask Wallet Connection

```

import { ethers } from 'ethers'

const CONTRACT_ADDRESS =
import.meta.env.VITE_CONTRACT_ADDRESS
const ETHERSCAN_BASE = import.meta.env.VITE_ETHERSCAN_BASE ||
'https://sepolia.etherscan.io/tx/'

const ABI = [

```

```

'function recordDonation(uint256 donationId, address donor, address hospital,
bytes32 donationHash, string nftTokenURI) external',
'event DonationRecorded(uint256 indexed donationId, address indexed donor,
address indexed hospital, bytes32 donationHash, uint256 bdcAwarded, uint256
nftTokenId, uint256 timestamp)',
]

```

```

// CONNECT WALLET

```

```

export async function connectWallet() {
  if (!window.ethereum) throw new Error('MetaMask not installed')
  const provider = new ethers.BrowserProvider(window.ethereum)
  await provider.send('eth_requestAccounts', [])
  const signer = await provider.getSigner()
  const address = await signer.getAddress()
  return { provider, signer, address }
}

```

```

// GET WALLET ADDRESS

```

```

export async function getWalletAddress() {
  if (!window.ethereum) return null
  const accounts = await new
ethers.BrowserProvider(window.ethereum).listAccounts()
  return accounts[0]?.address || null
}

```

```

// COMPUTE HASH

```

```

export function computeDonationHash(bloodBagId, donorWallet, isoTimestamp) {

```

```

    return
    ethers.keccak256(ethers.toUtf8Bytes(`${bloodBagId}${donorWallet.toLowerCase()
    }${isoTimestamp}`))
  }

```

// RECORD ON CHAIN

```

export async function recordOnChain({ donationId, bloodBagId, donorAddress,
hospitalAddress, donorConfirmedAt, nftTokenURI = 'ipfs://placeholder' }) {
  if (!CONTRACT_ADDRESS) throw new Error('Contract address not
configured')

  const { signer } = await connectWallet()
  const contract = new ethers.Contract(CONTRACT_ADDRESS, ABI, signer)
  const donationHash = computeDonationHash(bloodBagId, donorAddress, new
Date(donorConfirmedAt).toISOString())

  const idBig = BigInt(donationId.replace(/^[0-9]/g, "").slice(0, 15) || '1')
  const receipt = await (await contract.recordDonation(idBig, donorAddress,
hospitalAddress, donationHash, nftTokenURI)).wait()

  return { txHash: receipt.hash, blockchainHash: donationHash, etherscanUrl:
`${ETHERSCAN_BASE}${receipt.hash}` }
}

```

// TRUNCATE ADDRESS

```

export const truncateAddress = (addr) => addr ? `${addr.slice(0,
6)}...${addr.slice(-4)}` : ""

```

2. Donor Dashboard Page

```
import { useState, useEffect } from 'react'
import { useAuthStore } from '../store/authStore'
import { useRequestStore } from '../store/requestStore'
import { getMyDonations, donorConfirm, uploadProof } from '../api/donations'
import { BloodBadge, UrgencyChip, StatusBadge, StatusStep, BDCCounter,
EmptyState, CardSkeleton, ConfirmDialog, TxHashBadge } from
'../components/ui/index'
import { formatDate } from '../utils/formatters'
import toast from 'react-hot-toast'
import { useNavigate } from 'react-router-dom'
import { MapPin, Clock, Droplets } from 'lucide-react'
import DonorMap from '../components/donor/DonorMap'
import PublicInventoryPage from './PublicInventoryPage'

// REQUEST CARD
function RequestCard({ req, onAccept }) {
  const [confirming, setConfirming] = useState(false)
  const [loading, setLoading] = useState(false)
  const handleAccept = async () => {
    setLoading(true)
    try { await onAccept(req._id) }
    finally { setLoading(false); setConfirming(false) }
  }
  return (
    <>
    <div className="card hover:border-slate-700 transition-all animate-fade-in">
```

```

<div className="flex items-start justify-between mb-3">
  <BloodBadge group={req.bloodGroup} />
  <UrgencyChip level={req.urgencyLevel} />
</div>

<h3 className="font-medium text-slate-100 mb-1
truncate">{req.hospital?.hospitalName || req.hospital?.name}</h3>

  <div className="flex items-center gap-1.5 text-slate-400 text-xs mb-
1"><MapPin size={12}/>{req.location?.city}</div>

  <div className="flex items-center gap-1.5 text-slate-400 text-xs mb-
4"><Droplets size={12}/>{req.unitsRequired} unit{req.unitsRequired > 1 ? 's' : ''}
needed</div>

  {req.notes && <p className="text-xs text-slate-500 mb-4 line-clamp-
2">{req.notes}</p>}}

  <div className="flex items-center justify-between">
    <span className="text-xs text-slate-600"><Clock size={10}
className="inline mr-1"/>{formatDate(req.createdAt)}</span>

    <button onClick={() => setConfirming(true)} className="btn-primary text-
xs px-3 py-1.5">Accept Request</button>
  </div>
</div>

<ConfirmDialog open={confirming} title="Accept Blood Request?"
message={`Committing to donate ${req.bloodGroup} to
${req.hospital?.hospitalName || 'this hospital'}.`}
onConfirm={handleAccept} onCancel={() => setConfirming(false)} danger
/>
</>
)

```

```
}
```

```
// MY DONATIONS TABLE
```

```
function MyDonationsTable() {
```

```
  const [donations, setDonations] = useState([])
```

```
  const [loading, setLoading] = useState(true)
```

```
  const navigate = useNavigate()
```

```
  const load = async () => {
```

```
    try { const res = await getMyDonations(); setDonations(res.data.data) }
```

```
    catch { toast.error('Failed to load donations') }
```

```
    finally { setLoading(false) }
```

```
  }
```

```
  useEffect(() => { load() }, [])
```

```
  const handleConfirm = async (id) => {
```

```
    try { await donorConfirm(id); toast.success('Confirmed!'); load() }
```

```
    catch (err) { toast.error(err.response?.data?.message || 'Failed') }
```

```
  }
```

```
  const handleUpload = async (id, file) => {
```

```
    try { await uploadProof(id, file); toast.success('Proof uploaded!'); load() }
```

```
    catch { toast.error('Upload failed') }
```

```
  }
```

```
  if (loading) return <div className="grid gap-3">{[1,2,3].map(i =>  
<CardSkeleton key={i}/>)}</div>
```

```

    if (!donations.length) return <EmptyState title="No donations yet"
description="Accept a blood request to start."/>

    return (
      <div className="overflow-x-auto rounded-xl border border-slate-800">
        <table className="w-full text-sm">
          <thead className="bg-slate-900 text-slate-400 text-xs uppercase tracking-
wider">
            <tr>{['Hospital','Blood','Date','Progress','Status','Actions'].map(h => <th
key={h} className="px-4 py-3 text-left">{h}</th>)}</tr>
            </thead>
            <tbody className="divide-y divide-slate-800">
              {donations.map((d) => (
                <tr key={d._id} className="hover:bg-slate-900/50 transition-colors">
                  <td className="px-4 py-3 text-slate-200 font-
medium">{d.hospital?.hospitalName || d.hospital?.name}</td>
                  <td className="px-4 py-3"><BloodBadge
group={d.request?.bloodGroup}/></td>
                  <td className="px-4 py-3 text-slate-400 text-
xs">{formatDate(d.createdAt)}</td>
                  <td className="px-4 py-3"><StatusStep status={d.status}/></td>
                  <td className="px-4 py-3"><StatusBadge status={d.status}/></td>
                  <td className="px-4 py-3">
                    <div className="flex gap-2 flex-wrap">
                      {d.status === 'PENDING' && (
                        <button onClick={() => handleConfirm(d._id)} className="btn-
primary text-xs py-1 px-2">Confirm Donation</button>

```

```

    })
    {!d.proofImageUrl && d.status !== 'COMPLETED' && (
      <label className="btn-secondary text-xs py-1 px-2 cursor-pointer">
        Upload Proof
        <input type="file" accept="image/*,.pdf" hidden onChange={(e) =>
e.target.files[0] && handleUpload(d._id, e.target.files[0])}/>
      </label>
    )}
    {d.readyForBlockchain && d.status !== 'COMPLETED' && (
      <button onClick={() => navigate(`/donations/${d._id}`)}
className="bg-purple-700 hover:bg-purple-600 text-white text-xs py-1 px-2
rounded-lg">⌘Record On-Chain</button>
    )}
    {d.txHash && <TxHashBadge hash={d.txHash}/>}
  </div>
</td>
</tr>
)}}
</tbody>
</table>
</div>
)
}

```

// DONOR DASHBOARD

```

export default function DonorDashboard() {
  const { user } = useAuthStore()

```

```

const { requests, loading, fetchRequests } = useRequestStore()
const [tab, setTab] = useState('requests')
const [bgFilter, setBg] = useState("")
const [urgFilter, setUrg] = useState("")

useEffect(() => { fetchRequests({ status: 'OPEN' }) }, [])

const handleAccept = async (id) => {
  const { acceptRequest } = await import('../api/requests')
  try { const res = await acceptRequest(id); toast.success('Accepted!');
fetchRequests({ status: 'OPEN' }); return res }
  catch (err) { toast.error(err.response?.data?.message || 'Could not accept') }
}

const filtered = requests.filter(r =>
  (!bgFilter || r.bloodGroup === bgFilter) &&
  (!urgFilter || r.urgencyLevel === urgFilter)
)

const TABS = [
  { id: 'requests', label: `Available (${requests.filter(r => r.status ===
'OPEN').length})` },
  { id: 'donations', label: 'My Donations' },
  { id: 'inventory', label: '🩸 Blood Availability' },
  { id: 'map', label: 'Map View' },
]

```

```

return (
  <div className="animate-fade-in">
    <div className="flex items-center justify-between mb-6">
      <div>
        <h2 className="text-2xl font-display text-slate-100">WELCOME,
{user?.name?.split(' ')[0]?.toUpperCase()}</h2>
        <p className="text-slate-500 text-sm mt-0.5">Blood Group: <span
className="text-slate-300 font-medium">{user?.bloodGroup}</span></p>
      </div>
      <BDCCounter balance={user?.bdcBalance || 0}/>
    </div>

    <div className="flex gap-1 bg-slate-900 rounded-xl p-1 mb-6 w-fit border
border-slate-800 flex-wrap">
      {TABS.map(({ id, label }) => (
        <button key={id} onClick={() => setTab(id)}
          className={`px-4 py-2 rounded-lg text-sm font-medium transition-all
${tab === id ? 'bg-blood-600 text-white' : 'text-slate-400 hover:text-slate-200'} `}>
          {label}
        </button>
      ))}
    </div>

    {tab === 'requests' && (
      <>
        <div className="flex flex-wrap gap-3 mb-5">

```

```

        <select className="input w-auto text-sm" value={bgFilter} onChange={e
=> setBg(e.target.value)}>
            <option value="">All Blood Groups</option>
            {[ 'A+', 'A-', 'B+', 'B-', 'O+', 'O-', 'AB+', 'AB-' ].map(g => <option
key={g}>{g}</option>)}
        </select>

        <select className="input w-auto text-sm" value={urgFilter} onChange={e
=> setUrg(e.target.value)}>
            <option value="">All Urgency</option>
            {[ 'Critical', 'High', 'Medium' ].map(u => <option key={u}>{u}</option>)}
        </select>

        {(bgFilter || urgFilter) && <button className="btn-ghost text-xs"
onClick={() => { setBg(""); setUrg("") }}>Clear filters</button>}

    </div>

    {loading
      ? <div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-
4">{[1,2,3,4,5,6].map(i => <CardSkeleton key={i}/>)}</div>
      : filtered.length === 0
      ? <EmptyState icon="🔍" title="No open requests" description="No
requests match your filters."/>
      : <div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-
4">{filtered.map(r => <RequestCard key={r._id} req={r}
onAccept={handleAccept}/>)}</div>
    }
  </>
)}

{tab === 'donations' && <MyDonationsTable/>}

```

```

{tab === 'inventory' && <PublicInventoryPage/>}
{tab === 'map' && <DonorMap/>} </div>))

```

3. Hospital Dashboard Page

```

import { useState, useEffect } from 'react'
import { getHospitalStats, getHospitalDonations } from '../api/admin'
import { createRequest } from '../api/requests'
import { receiverConfirm } from '../api/donations'
import { BloodBadge, StatusBadge, TxHashBadge, EmptyState, LoadingSpinner }
from '../components/ui/index'
import { formatDate } from '../utils/formatters'
import toast from 'react-hot-toast'
import { useNavigate } from 'react-router-dom'
import { Plus, X } from 'lucide-react'
import { useAuthStore } from '../store/authStore'
import HospitalInventoryPage from './HospitalInventoryPage'

```

```

const BLOOD_GROUPS = ['A+', 'A-', 'B+', 'B-', 'O+', 'O-', 'AB+', 'AB-']
const URGENCY = ['Critical', 'High', 'Medium']

```

```
// STAT CARD
```

```

function StatCard({ label, value, color = 'text-slate-100' }) {
  return (
    <div className="card text-center">
      <p className={`font-display text-4xl ${color} mb-1`} >{value ?? '—'}</p>
      <p className="text-slate-500 text-xs uppercase tracking-wider">{label}</p>
    </div>
  )
}

```

```
)  
}
```

```
// CREATE REQUEST MODAL
```

```
function CreateRequestModal({ onClose, onSuccess }) {  
  const [form, setForm] = useState({ bloodGroup: 'O+', unitsRequired: 1,  
urgencyLevel: 'High', city: 'Chennai', lat: '13.0625', lng: '80.2707', notes: " })  
  const [loading, setLoading] = useState(false)  
  const set = (k, v) => setForm(p => ({ ...p, [k]: v })))  
  
  const handleSubmit = async (e) => {  
    e.preventDefault(); setLoading(true)  
    try {  
      await createRequest({ ...form, lat: parseFloat(form.lat), lng:  
parseFloat(form.lng), unitsRequired: parseInt(form.unitsRequired) })  
      toast.success('Blood request posted!'); onSuccess(); onClose()  
    } catch (err) { toast.error(err.response?.data?.message || 'Failed') }  
    finally { setLoading(false) }  
  }  
  
  return (  
    <div className="fixed inset-0 z-50 flex items-center justify-center bg-black/70  
backdrop-blur-sm">  
      <div className="card w-full max-w-lg mx-4 border-slate-700 shadow-2xl">  
        <div className="flex justify-between items-center mb-5">  
          <h3 className="font-display text-2xl text-slate-100">POST BLOOD  
REQUEST</h3>
```

```

    <button onClick={onClose} className="btn-ghost p-1"><X
size={18}/></button>
  </div>
  <form onSubmit={handleSubmit} className="space-y-4">
    <div className="grid grid-cols-2 gap-3">
      <div><label className="label">Blood Group</label>
      <select className="input" value={form.bloodGroup} onChange={e =>
set('bloodGroup', e.target.value)}>
        {BLOOD_GROUPS.map(g => <option key={g}>{g}</option>)}
      </select>
    </div>
    <div><label className="label">Units Required</label>
    <input type="number" className="input" min="1" max="20"
value={form.unitsRequired} onChange={e => set('unitsRequired',
e.target.value)}/>
    </div>
    <div><label className="label">Urgency</label>
    <select className="input" value={form.urgencyLevel} onChange={e =>
set('urgencyLevel', e.target.value)}>
      {URGENCY.map(u => <option key={u}>{u}</option>)}
    </select>
    </div>
    <div><label className="label">City</label>
    <input className="input" value={form.city} onChange={e => set('city',
e.target.value)} required/>
    </div>
    <div><label className="label">Latitude</label>

```

```

        <input type="number" step="any" className="input" value={form.lat}
onChange={e => set('lat', e.target.value)}/>
    </div>
    <div><label className="label">Longitude</label>
        <input type="number" step="any" className="input" value={form.lng}
onChange={e => set('lng', e.target.value)}/>
    </div>
</div>
<div><label className="label">Notes (optional)</label>
    <textarea className="input resize-none" rows={2} value={form.notes}
onChange={e => set('notes', e.target.value)}/>
</div>
<div className="flex gap-3 pt-2">
    <button type="button" onClick={onClose} className="btn-secondary
flex-1">Cancel</button>
    <button type="submit" disabled={loading} className="btn-primary flex-
1">{loading ? 'Posting...' : 'Post Request'}</button>
</div>
</form>
</div>
</div>
)
}

```

// INCOMING DONATIONS TABLE

```

function IncomingDonationsTable() {
    const [donations, setDonations] = useState([])

```

```

const [loading, setLoading] = useState(true)
const [bagIds, setBagIds] = useState({})
const navigate = useNavigate()

const load = async () => {
  setLoading(true)
  try { const res = await getHospitalDonations(); setDonations(res.data.data) }
  catch { toast.error('Failed to load donations') }
  finally { setLoading(false) }
}
useEffect(() => { load() }, [])

const handleConfirmReceipt = async (id) => {
  const bloodBagId = bagIds[id]?.trim()
  if (!bloodBagId) return toast.error('Enter Blood Bag ID first')
  try { await receiverConfirm(id, bloodBagId); toast.success('Receipt
confirmed!'); load() }
  catch (err) { toast.error(err.response?.data?.message || 'Failed') }
}

if (loading) return <LoadingSpinner className="py-12"/>
if (!donations.length) return <EmptyState icon="📭" title="No incoming
donations" description="Donations appear when donors accept your requests."/>

return (
  <div className="overflow-x-auto rounded-xl border border-slate-800">
    <table className="w-full text-sm">

```

```

    <thead className="bg-slate-900 text-slate-400 text-xs uppercase tracking-
wider">
      <tr>{['Donor','Blood','Status','Blood Bag ID','Actions'].map(h => <th
key={h} className="px-4 py-3 text-left">{h}</th>)}</tr>
    </thead>
    <tbody className="divide-y divide-slate-800">
      {donations.map(d => (
        <tr key={d._id} className="hover:bg-slate-900/50 transition-colors">
          <td className="px-4 py-3">
            <p className="text-slate-200 font-medium">{d.donor?.name}</p>
            <p className="text-xs text-slate-500 font-
mono">{d.donor?.walletAddress?.slice(0,10)}...</p>
          </td>
          <td className="px-4 py-3"><BloodBadge
group={d.request?.bloodGroup}/></td>
          <td className="px-4 py-3"><StatusBadge status={d.status}/></td>
          <td className="px-4 py-3">
            {d.receiverConfirmed
              ? <span className="text-xs font-mono text-slate-
400">{d.bloodBagId}</span>
                : <input className="input text-xs py-1.5 w-40" placeholder="BAG-
TN-001"
                  value={bagIds[d._id] || ""} onChange={e => setBagIds(p => ({ ...p,
[d._id]: e.target.value })))}/>
              }
            </td>
          <td className="px-4 py-3">

```

```

    <div className="flex gap-2 flex-wrap">
      {!d.receiverConfirmed && <button onClick={() =>
handleConfirmReceipt(d._id)} className="btn-primary text-xs py-1 px-
2">Confirm Receipt</button>}
      {d.readyForBlockchain && d.status !== 'COMPLETED' && (
        <button onClick={() => navigate(`/donations/${d._id}`)}
className="bg-purple-700 hover:bg-purple-600 text-white text-xs py-1 px-2
rounded-lg">📄Record</button>
      )}
      {d.txHash && <TxHashBadge hash={d.txHash}/>}
    </div>
  </td>
</tr>
)}}
</tbody>
</table>
</div>
)
}

```

// HOSPITAL DASHBOARD

```

export default function HospitalDashboard() {
  const { user } = useAuthStore()
  const [stats, setStats] = useState(null)
  const [showModal, setModal] = useState(false)
  const [tab, setTab] = useState('donations')

  const loadStats = async () => {

```

```

    try { const res = await getHospitalStats(); setStats(res.data) } catch {}
  }
  useEffect(() => { loadStats() }, [])

  const TABS = [
    { id: 'donations', label: '📄 Incoming Donations' },
    { id: 'inventory', label: '📄 Blood Inventory' },
  ]

  return (
    <div className="animate-fade-in space-y-6">
      <div className="grid grid-cols-2 lg:grid-cols-4 gap-4">
        <StatCard label="Open Requests" value={stats?.requests?.open}
color="text-blood-400"/>
        <StatCard label="Active Donations" value={({stats?.donations?.pending || 0) +
(stats?.donations?.donorConfirmed || 0)} color="text-blue-400"/>
        <StatCard label="Completed" value={stats?.donations?.completed}
color="text-green-400"/>
        <StatCard label="BDC Issued" value={stats?.totalBDCIssued}
color="text-yellow-400"/>
      </div>

      <div className="flex items-center justify-between flex-wrap gap-3">
        <div className="flex gap-1 bg-slate-900 rounded-xl p-1 border border-slate-
800">
          {TABS.map(({ id, label }) => (
            <button key={id} onClick={() => setTab(id)}

```

```

        className={`px-4 py-2 rounded-lg text-sm font-medium transition-all
${tab === id ? 'bg-blood-600 text-white' : 'text-slate-400 hover:text-slate-200'}`}>
        {label}
      </button>
    )}
  </div>

  {tab === 'donations' && (
    <button onClick={() => setModal(true)} className="btn-primary flex
items-center gap-2"><Plus size={16}/> Post Blood Request</button>
  )}
</div>

{tab === 'donations' && <IncomingDonationsTable/>}
{tab === 'inventory' && <HospitalInventoryPage hospitalId={user?._id ||
user?.id}/> }

{showModal && <CreateRequestModal onClose={() => setModal(false)}
onSuccess={loadStats}/> }
</div>
)
}

```

4. AI Analyser — Live Data Builder and Chat Endpoint

```

import express from 'express'
import Groq from 'groq-sdk'
import User from '../models/User.js'
import Donation from '../models/Donation.js'

```

```

import BloodRequest from '../models/BloodRequest.js'
import BloodInventory from '../models/BloodInventory.js'
import InterHospitalTransfer from '../models/InterHospitalTransfer.js'
import BDCLedger from '../models/BDCLedger.js'
import { protect } from '../middleware/auth.js'

const router = express.Router()
const groq = new Groq({ apiKey: process.env.GROQ_API_KEY })
const GROQ_MODEL = 'llama-3.3-70b-versatile'

// Build role-specific live data snapshot from MongoDB
async function buildLiveData(user) {
  if (user.role === 'ADMIN') {
    const [users, donations, requests, transfers, bdcLedger, inventory] =
      await Promise.all([
        User.find().select('-password').lean(),
        Donation.find()
          .populate('donor', 'name email bloodGroup city bdcBalance')
          .populate('hospital', 'name city').lean(),
        BloodRequest.find().populate('hospital', 'name city').lean(),
        InterHospitalTransfer.find()
          .populate('requestingHospital', 'name city')
          .populate('supplyingHospital', 'name city').lean(),
        BDCLedger.find().lean(),
        BloodInventory.find().populate('hospital', 'name city').lean(),
      ])
  }
}

```

```

    ])
    return { role: 'ADMIN', users, donations, requests, transfers,
            bdcLedger, inventory }
}

if (user.role === 'HOSPITAL') {
    const [hDonations, hRequests, inventory, transfers] = await Promise.all([
        Donation.find({ hospital: user._id })
            .populate('donor', 'name email bloodGroup city bdcBalance
                    isVerified age weight').lean(),
        BloodRequest.find({ hospital: user._id }).lean(),
        BloodInventory.findOne({ hospital: user._id }).lean(),
        InterHospitalTransfer.find({
            $or: [{ requestingHospital: user._id },
                  { supplyingHospital: user._id }]
        }).lean(),
    ])
    return { role: 'HOSPITAL', hDonations, hRequests, inventory, transfers }
}

```

// DONOR

```

const [myDonations, openRequests, bdcLedger] = await Promise.all([
    Donation.find({ donor: user._id })
        .populate('hospital', 'name city').lean(),
    BloodRequest.find({ status: 'OPEN' })

```

```

        .populate('hospital', 'name city').lean(),
        BDCLedger.find({ user: user._id }).lean(),
    ])
    return { role: 'DONOR', myDonations, openRequests, bdcLedger }
}

// POST /api/ai/chat — conversational AI with live data
router.post('/chat', protect, async (req, res) => {
  try {
    const { messages = [] } = req.body
    const data = await buildLiveData(req.user)
    const systemPrompt =
      `You are BloodLink AI. Role: ${data.role}. Be concise and
      actionable. Highlight critical issues with ☐.
      LIVE DATA: ${JSON.stringify(data)}`
    const completion = await groq.chat.completions.create({
      model: GROQ_MODEL,
      messages: [{ role: 'system', content: systemPrompt }, ...messages],
      temperature: 0.4,
      max_tokens: 1500,
    })
    res.json({ success: true,
      text: completion.choices[0]?.message?.content })
  } catch (e) {
    res.status(500).json({ success: false, message: e.message })
  }
})

```

```

    }
  })

// POST /api/ai/analyse — deep topic analysis
router.post('/analyse', protect, async (req, res) => {
  try {
    const { topic = 'overall platform status' } = req.body
    const data = await buildLiveData(req.user)
    const completion = await groq.chat.completions.create({
      model: GROQ_MODEL,
      messages: [
        { role: 'system', content:
          `You are BloodLink AI analyst. Analyse platform data deeply.
          DATA: ${JSON.stringify(data)} `,
        { role: 'user', content: `Topic: ${topic} `,
      ],
      temperature: 0.3,
      max_tokens: 2000,
    })
    res.json({ success: true,
      analysis: completion.choices[0]?.message?.content })
  } catch (e) {
    res.status(500).json({ success: false, message: e.message })
  }
})

```

```
// POST /api/ai/report — structured report generation
router.post('/report', protect, async (req, res) => {
  try {
    const { reportType = 'platform_health' } = req.body
    const data = await buildLiveData(req.user)
    const prompts = {
      platform_health: 'Write an executive platform health report in
        markdown covering Users, Donations, Requests, BDC Economy,
        and Blockchain Compliance.',
      blood_supply: 'Write a blood supply and demand analysis with
        per-blood-group tables and shortage alerts.',
      donor_engagement: 'Write a donor engagement report with
        segmentation and re-engagement strategies.',
      hospital_performance: 'Write a hospital performance ranking
        report by completion rate and dispute rate.',
      compliance: 'Write a blockchain proof compliance report with
        non-compliant donation identification.',
    }
    const completion = await groq.chat.completions.create({
      model: GROQ_MODEL,
      messages: [
        { role: 'system', content:
          `You are BloodLink AI. Generate professional reports from
            real data. DATA: ${JSON.stringify(data)} ` },

```

```

    { role: 'user',
      content: prompts[reportType] || prompts.platform_health },
    ],
    temperature: 0.3,
    max_tokens: 3000,
  })
  res.json({ success: true,
    report: completion.choices[0]?.message?.content,
    reportType, generatedAt: new Date() })
} catch (e) {
  res.status(500).json({ success: false, message: e.message })
}
})export default router

```

D. Output Description

When blood availability is normal:

- Donor registered with matching blood group
- No active blood requests in the area
- Dashboard shows no pending notifications

When a blood request is posted:

- Hospital posts request with blood group and location
- System identifies eligible donors within 50 km radius
- Matched donors receive immediate email notification
- Donor dashboard displays new request on interactive map

When donation is completed:

- Donor confirms availability and uploads blood test report
- Hospital confirms receipt of donation
- System awards 100 BDC tokens to donor's account
- Donation hash is recorded on Ethereum Sepolia blockchain
- Both donor and hospital receive completion email notification
- Blood inventory is automatically updated in hospital dashboard

When AI Analyser is used:

- Admin queries the AI chat and receives a real-time platform health summary with shortage alerts and hospital rankings
- Hospital receives AI-generated inventory alerts and pending confirmation summaries
- Donor receives personalised eligibility status and compatible nearby blood request suggestions
- Admin generates structured reports (platform health, blood supply, donor engagement) from live data in seconds

CHAPTER 11

SCREENSHOTS

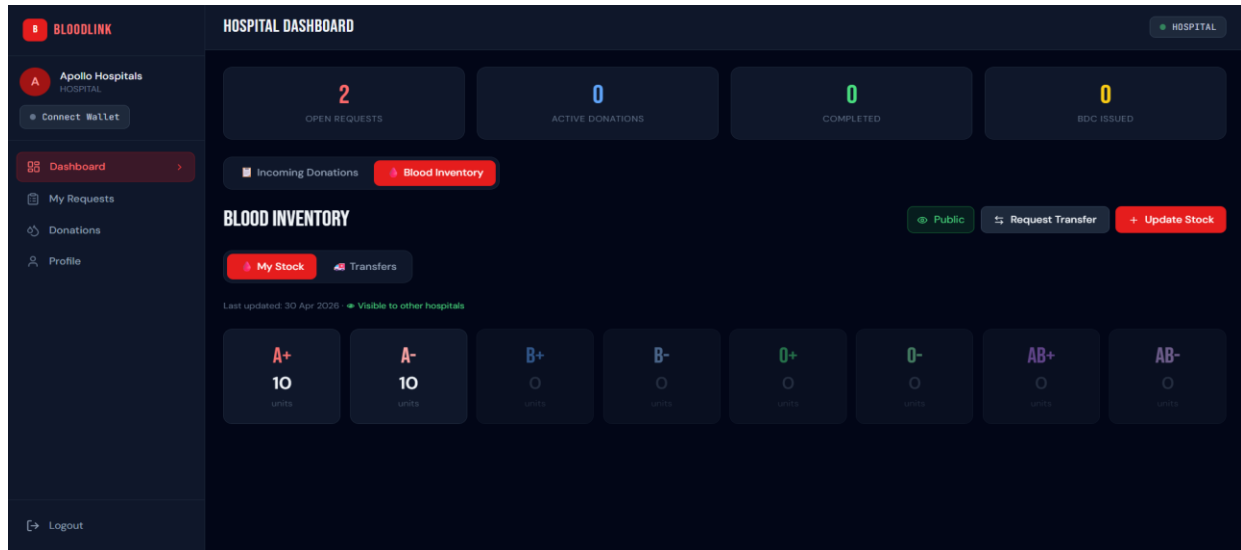


Fig 1: Blood Inventory

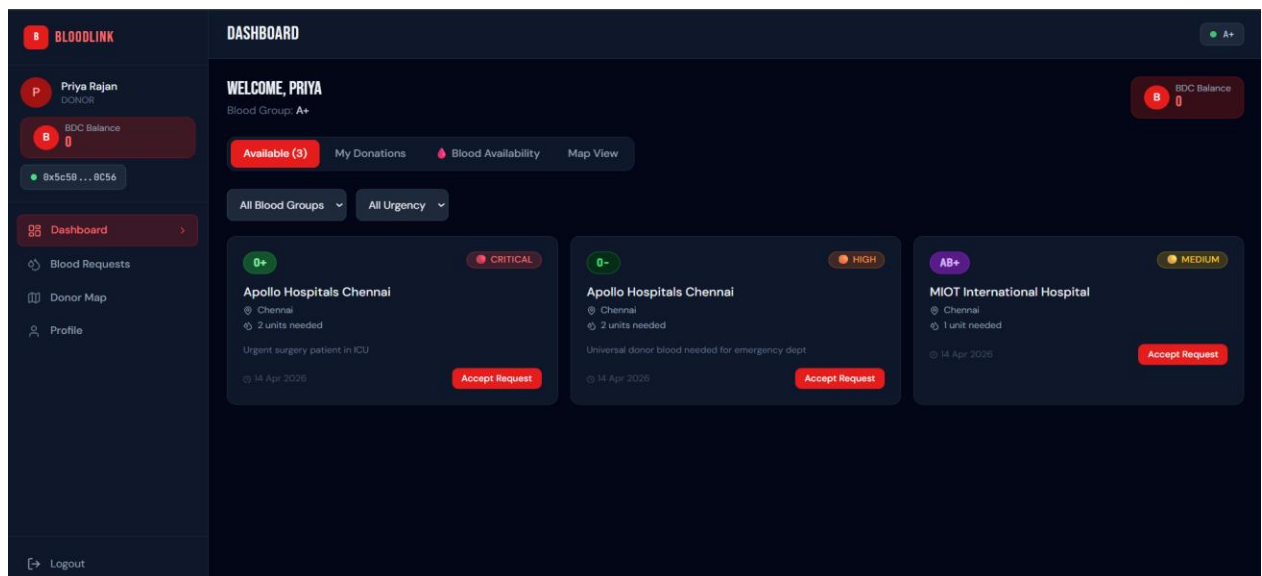


Fig 2: Donor Dashboard

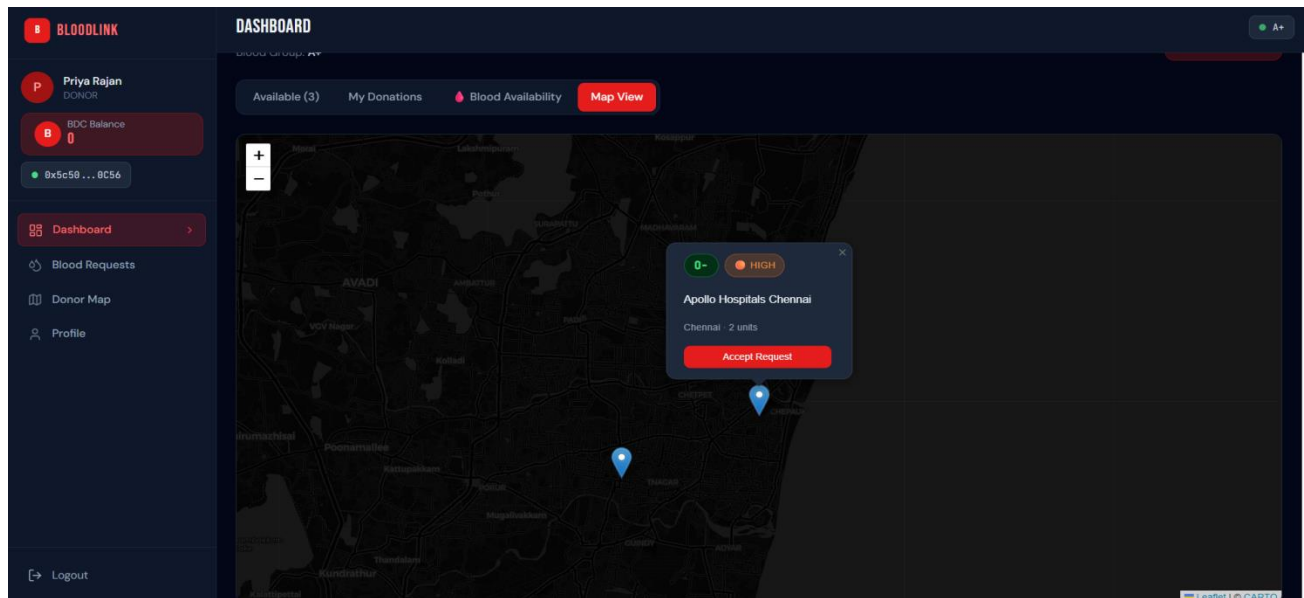


Fig 3: Map View

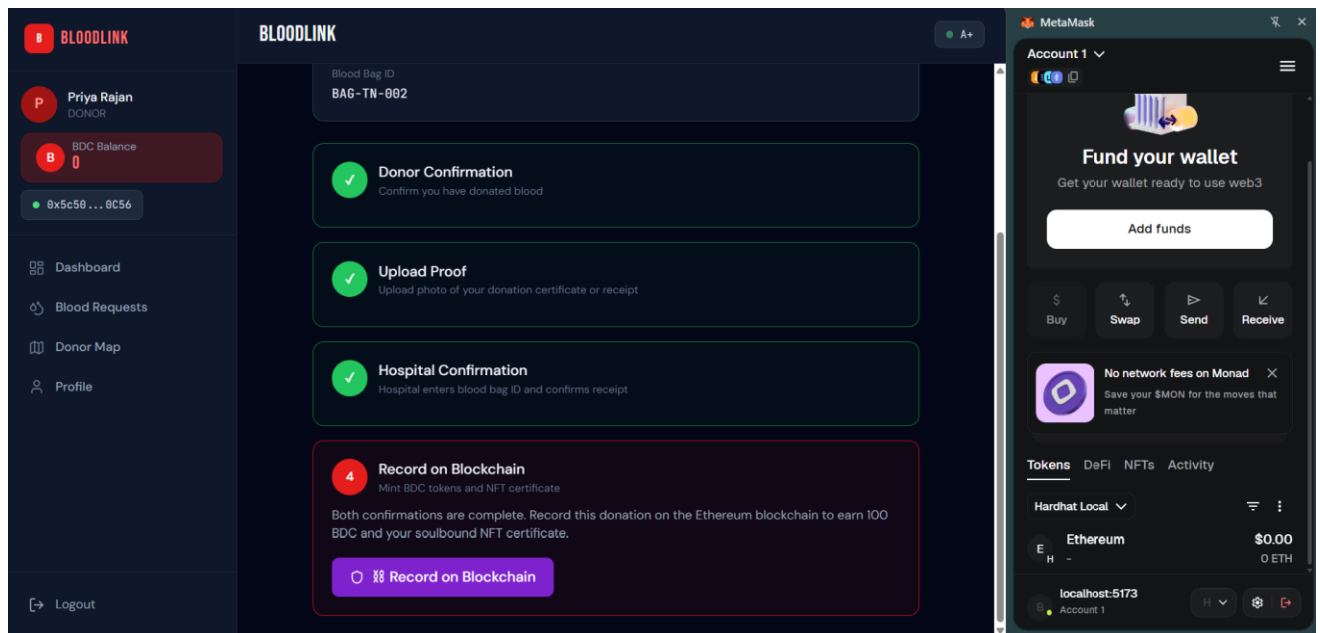


Fig 4: Blockchain Recording

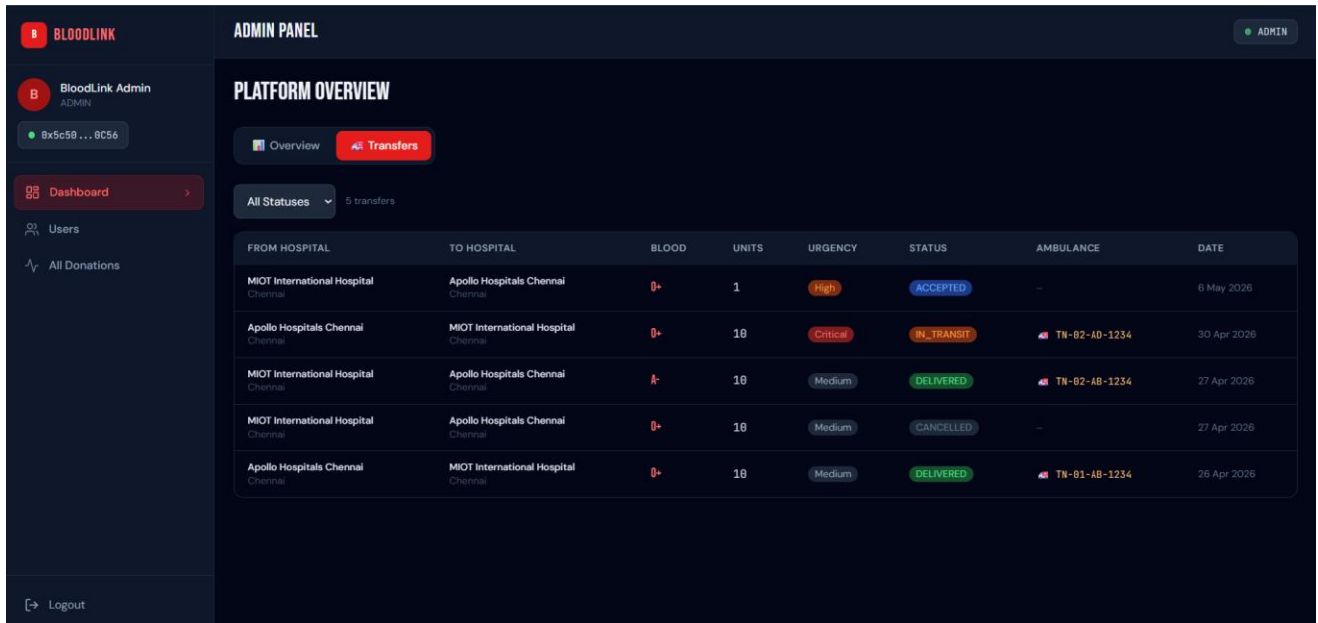


Fig 5: Admin Panel

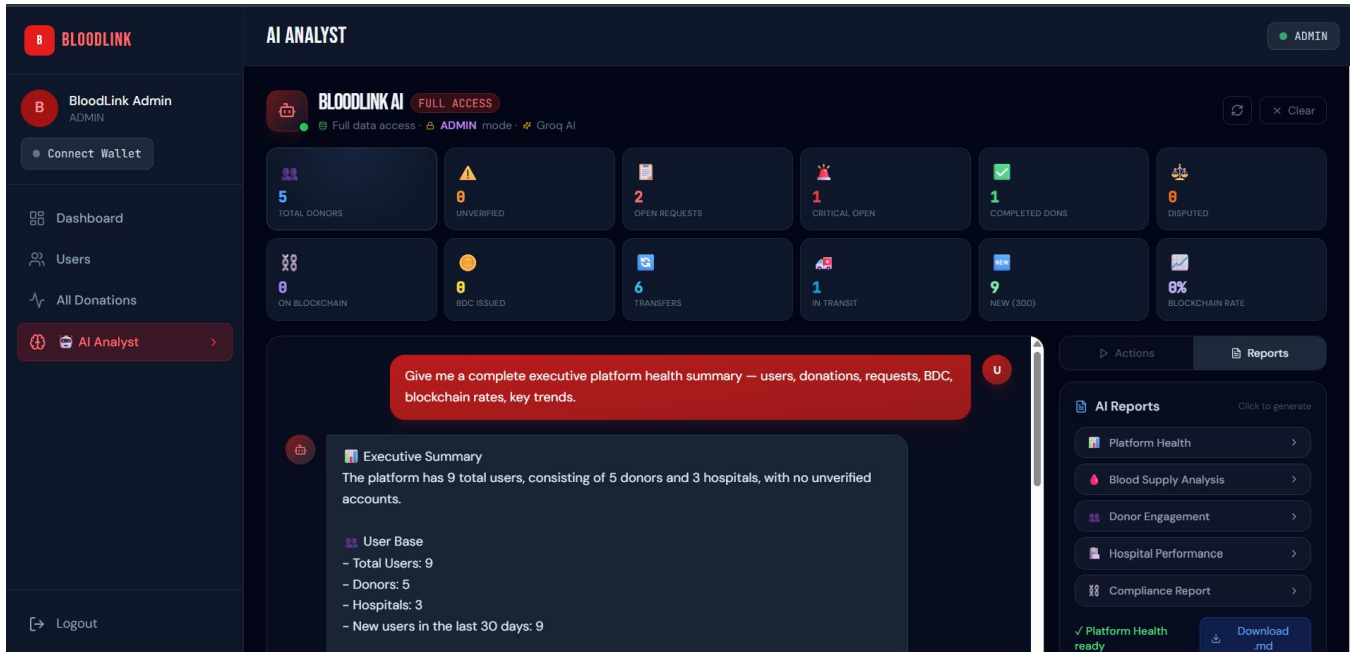


Fig 6: AI Analyst Dashboard

CHAPTER 12

REFERENCES

React, "React – A JavaScript Library for Building User Interfaces," Available: <https://react.dev>

Node.js, "Node.js Documentation," Available: <https://nodejs.org>

MongoDB, "MongoDB Manual – Geospatial Queries and Indexing," Available: <https://www.mongodb.com/docs>

Ethereum Foundation, "Ethereum Developer Documentation and Sepolia Testnet," Available: <https://ethereum.org/en/developers>

ethers.js, "ethers.js – Ethereum JavaScript Library Documentation," Available: <https://docs.ethers.org>

Cloudinary, "Cloudinary Media Management Platform Documentation," Available: <https://cloudinary.com/documentation>

Agbo, C. C., Mahmoud, Q. H., & Eklund, J. M., "Blockchain Technology in Healthcare: A Systematic Review," Healthcare, MDPI, vol. 7, no. 2, pp. 56, 2019.

Mettler, M., "Blockchain Technology in Healthcare: The Revolution Starts Here," Proceedings of IEEE 18th International Conference on e-Health Networking, Applications and Services, pp. 1–3, 2016.

Nugent, T., Upton, D., & Capper, M., "Improving Data Transparency in Clinical Trials Using Blockchain Smart Contracts," F1000Research, 2016.

Zhang, P., White, J., Schmidt, D. C., Lenz, G., & Rosenbloom, S. T., "FHIRChain: Applying Blockchain to Securely and Scalably Share Clinical Data," Computational and Structural Biotechnology Journal, vol. 16, pp. 267–278, 2018.

Figorilli, S., Antonucci, F., Costa, C., Pallottino, F., & Menesatti, P., "A Blockchain Implementation Prototype for the Electronic Open Source Traceability of Wood Along the Whole Supply Chain," Sensors, vol. 18, no. 9, pp. 3133, 2019.

National Blood Transfusion Council, India, "Annual Report on Blood Transfusion Services," Ministry of Health and Family Welfare, Available: <https://mohfw.gov.in>

Groq, "Groq API Documentation – LLaMA Model Inference," Available: <https://console.groq.com/docs>

Meta AI, "LLaMA 3: Open Foundation and Fine-Tuned Chat Models," Available: <https://ai.meta.com/llama>

CONFERENCE

Presented a paper titled “Decentralized Blood Donor Application” at the International Conference on Advanced Computing and Communication Technologies (ICACCT’26) organized by Francis Xavier Engineering College in association with VEI Technologies (March 2026).

CONFERENCE CERTIFICATE



FRANCIS XAVIER®
ENGINEERING COLLEGE
AN AUTONOMOUS INSTITUTION
Vannarpettai, Tirunelveli - 627 003.



An Autonomous Institution | Accredited by NBA(CSE, ECE, EEE, MECH) | AICTE Sponsored Margdarshan Mentor Institution
| DST-FIST Supported Institution | ISO 9001:2015 certified | Recognised Under Section 2(f) & 12(B) of the UGC Act, 1956 |
103/62, Bypass Road, Vannarpettai, Tirunelveli, Tamil Nadu, India - 627003.

DEPARTMENT OF INFORMATION TECHNOLOGY

IN ASSOCIATION WITH VEI TECHNOLOGIES

INTERNATIONAL CONFERENCE ON ADVANCED COMPUTING AND COMMUNICATION TECHNOLOGY
(ICACCT'26)

ISBN Number: 978-93-5779-500-5

THIS IS TO CERTIFY THAT Dr/ Mr/Ms/ Mrs. Lokesh V
PRESENTED A PAPER ENTITLED Decentralized blood donor application

IN THE INTERNATIONAL CONFERENCE ON ADVANCED COMPUTING AND COMMUNICATION TECHNOLOGY
(ICACCT'26) ORGANIZED BY DEPARTMENT OF INFORMATION TECHNOLOGY IN ASSOCIATION WITH
VEI TECHNOLOGIES HELD AT FRANCIS XAVIER ENGINEERING COLLEGE

ON 27th MARCH 2026.

Dr.B.EZHILAVAN
Convener
VEI Technologies

DR.M.CAROLINE VIOLA STELLA MARY
HoD | Convenor
Francis Xavier Engineering College

Dr.V.VELMURUGAN
Principal
Francis Xavier Engineering College



FRANCIS XAVIER®
ENGINEERING COLLEGE
AN AUTONOMOUS INSTITUTION
Vannarpettai, Tirunelveli - 627 003.



An Autonomous Institution | Accredited by NBA(CSE, ECE, EEE, MECH) | AICTE Sponsored Margdarshan Mentor Institution
| DST-FIST Supported Institution | ISO 9001:2015 certified | Recognised Under Section 2(f) & 12(B) of the UGC Act, 1956 |
103/62, Bypass Road, Vannarpettai, Tirunelveli, Tamil Nadu, India - 627003.

DEPARTMENT OF INFORMATION TECHNOLOGY

IN ASSOCIATION WITH VEI TECHNOLOGIES

INTERNATIONAL CONFERENCE ON ADVANCED COMPUTING AND COMMUNICATION TECHNOLOGY
(ICACCT'26)

ISBN Number: 978-93-5779-500-5

THIS IS TO CERTIFY THAT Dr/ Mr/Ms/ Mrs. Sasivarna Kumari S
PRESENTED A PAPER ENTITLED Decentralized blood donor application.

IN THE INTERNATIONAL CONFERENCE ON ADVANCED COMPUTING AND COMMUNICATION TECHNOLOGY
(ICACCT'26) ORGANIZED BY DEPARTMENT OF INFORMATION TECHNOLOGY IN ASSOCIATION WITH
VEI TECHNOLOGIES HELD AT FRANCIS XAVIER ENGINEERING COLLEGE

ON 27th MARCH 2026.

Dr.B.EZHILAVAN
Convener
VEI Technologies

DR.M.CAROLINE VIOLA STELLA MARY
HoD | Convenor
Francis Xavier Engineering College

Dr.V.VELMURUGAN
Principal
Francis Xavier Engineering College